Database Design

Nguyễn Văn Diêu

Ho Chi Minh City University of Transport

2026

Kiến thức - Kỹ năng - Sáng tạo - Hội nhập

Sứ mệnh - Tầm nhìn

Triết lý Giáo dục - Giá trị cốt lõi

Outline I

1. Overview of the Design Process

2. Conceptual Design

2.1 ER Conceptual Model
- A Sample Database Application
- Entity
- Relationship
- Constraints of relationships
- Patterns and Relationships
- Summary of the notation
- Database Example

2.2 ER Design Methodology

2.3 Example

2.4 Mapping to Relational Database
- Mapping Entity
- Mapping Relationship

2.5 Weak Entity

Outline II

- Strong Entity
- Weak entity and Identifying Relationship
- Weak Entity connected Weak Entity
- Update Methodology
- Mapping Weak Entity to Rational Database

2.6 Attributes of Relationship

2.7 Change M:N Relationship

2.8 Recursive Relationship
- Structural Constraints
- Multiple Relationship

2.9 Mapping Recursive Relationship

2.10 Derived or Redundant Relationship

2.11 Enhanced EntityRelationship (EER)
- Generalization & Specialization
- Subclasses of Subclasses

Outline III

- Mapping rules

2.12 Union Types
- Mapping rules

3. Logical Design

3.1 Functional Dependencies
- Definition
- Notation
- Satisfaction
- Integrity Constraint
- Inference

3.2 Amstrong's Axioms
- Amstrong's Axioms
- Lemma
- Proof

3.3 Closure

Outline IV

- Closure of FDs Set
- Closure of Attribute Set
- Algorithm
- Lemma
- Testing Membership
- Covers
- Minimum Covers
- Algorithm
- Projected FD
- Algorithm

3.4 Keys

- Definition
- Algorithm
- Remarkable
- Algorithm
- FDs by Keys

Outline V

3.5 Normal Form
- Definition
- 1NF
- Fully dependent
- Prime and nonprime attributes
- 2NF
- Transitively Dependent
- 3NF Classic
- 3NF Modern
- Lemma
- BCNF Classic
- BCNF Modern

3.6 Database Normalization
- Overview
- Preserve Information
- Tableau Test

Outline VI

- Theorem
- Preserve Dependencise
- Testing Preserve Dependencies
- Decomposition Algorithm
- Synthesis Algorithm

3.7 Multivalued Dependencies
- Overview
- Definition
- Lemma
- Trivial MVDs
- Theorem
- Test MVDs

3.8 Axioms for MVDs
- MVDs Alone
- FDs and MVD
- Finding X_F^+ by Chase

Outline VII

- Tableau Chase Test for MVDs
- Projecting MVDs
- Minimum Dependency Basis
- Beeri Algorithm

3.9 4NF
- Definition
- Lemma
- Decomposition into 4NF

3.10 Join Dependency
- Overview
- Definition
- Project Join NF
- Definition

3.11 Embedded Functional Dependencies
- Definition
- Embedded MVDs

Outline VIII

- Embedded JD

4. Physical Design

4.1 Overview

4.2 Conceptual to Physical
- Problem
- ERD
- Relational Model
- Normal Form
- Normalization
- Customize Name
- Foreign Keys
- SQL Script

4.3 Denormalization

4.4 Customized data views

4.5 Indexes

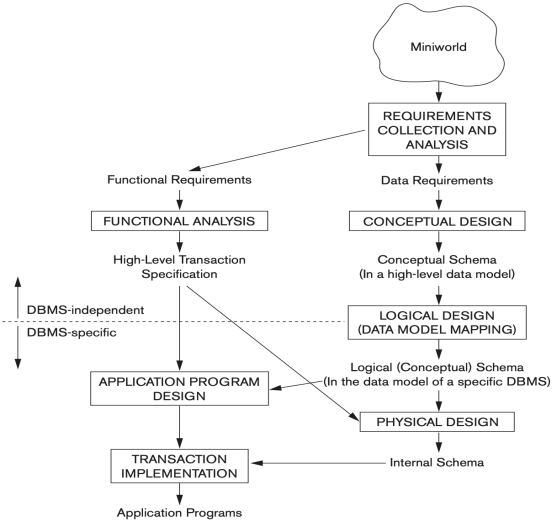
Outline IX

- Basic Concepts
- Common PostgreSQL Index Types
- Multicolumn Indexes
- Indexes and ORDER BY
- Combining Multiple Indexes
- Indexes on Expressions
- Partial Indexes
- Operator Classes
- Choosing the Right Index
- Best Practices for Indexing
- Common Index Summary

4.6 Query Processing
- Measures of Query Cost

4.7 Query Optimization
- Transformation of Relational Expressions
- Examples of Transformations

Main phases of Database design



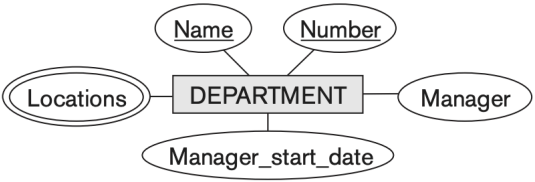
Company Database sample

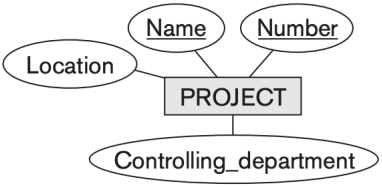
- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- The database will store each employee's name, Social Security number, address, salary, sex (gender), and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. It is required to keep track of the current number of hours per week that an employee works on each project, as well as the direct supervisor of each employee (who is another employee).
- The database will keep track of the dependents of each employee for insurance purposes, including each dependent's first name, sex, birth date, and relationship to the employee.

ER Conceptual Model

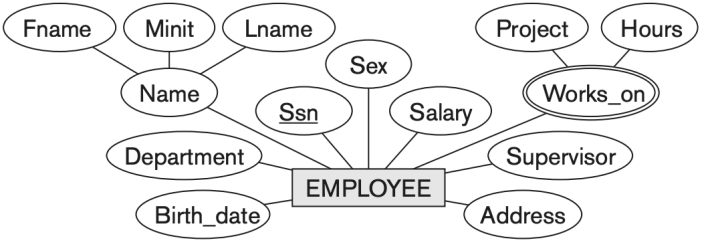
- **Entity**: a thing or object in the real world with an independent existence.
- **Attributes**: Each entity has attributes — the particular properties that describe it
- **Composite versus Simple (Atomic) Attributes**
- **Single-Valued versus Multivalued Attributes**
- **Stored versus Derived Attributes**
- **NULL Values**
- **Complex Attributes**
- **Key Attributes**

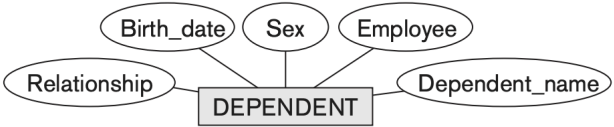
ER Conceptual Model





ER Conceptual Model





Relationship

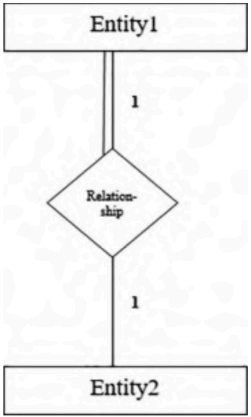
- **Relationship**: Whenever an attribute of one entity type refers to another entity type, some relationship exists.
- **Degree of a Relationship**: is the number of participating entity.
  binary is degree two,
  ternary is a degree three.
- **Attributes of Relationship**
- **Recursive Relationships**

Cardinality Ratio of a Relationship

- **One-to-One (1:1)** one entity is associated with one other entity and vice versa
- **Many-to-One (M:1)** many entity are associated with one another entity
- **One-to-Many (1:M)**
- **Many-to-Many (M:N)**
- **Participation (Full/Partial)**
  Full (doube lines): All entity associated with another entity
  Partial (single line): Not every entity associated with another entity

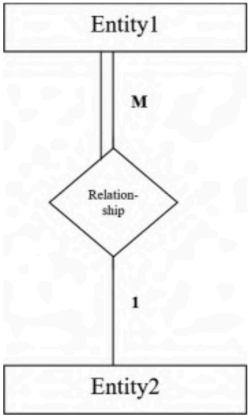
Pattern 1

1(full):1 Relationship



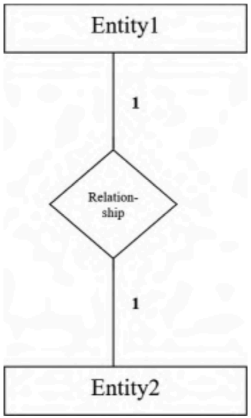
Pattern 1

M(full):1 Relationship



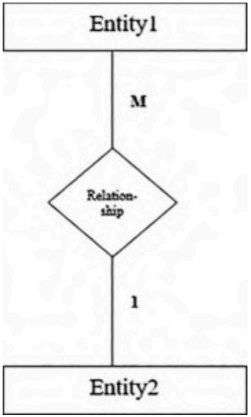
Pattern 2

1(partial):1 Relationship



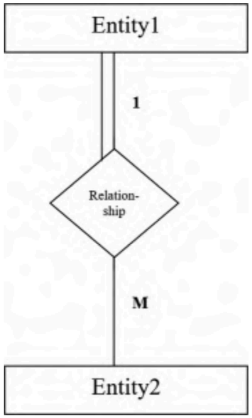
Pattern 2

M(partial):1 Relationship



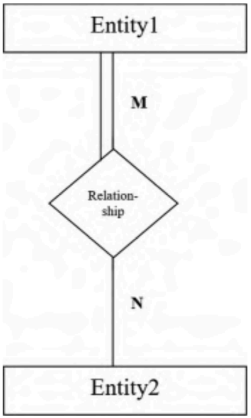
Pattern 3

1(full):M Relationship



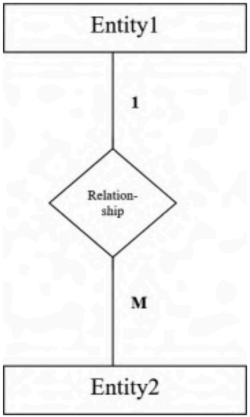
Pattern 3

M(full):N Relationship



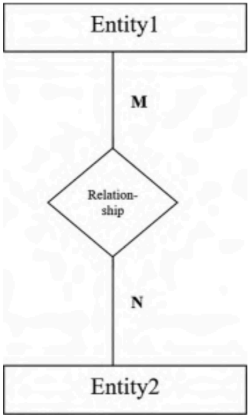
Pattern 4

1(partial):M Relationship



Pattern 4

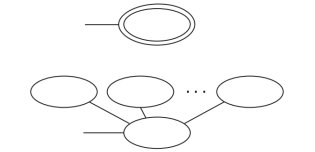
M(partial):N Relationship



Summary of the notation for ER diagrams

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Indentifying Relationship
	Attribute
	Key Attribute

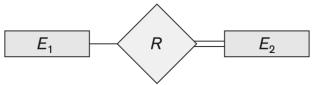
Summary of the notation for ER diagrams



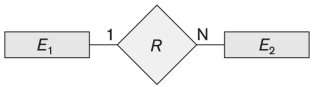
Multivalued Attribute

Composite Attribute

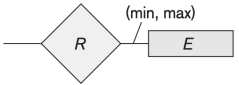
Derived Attribute



Total Participation of E_2 in R

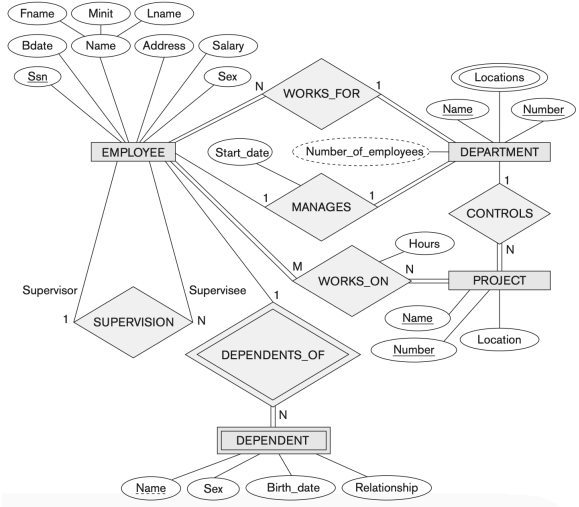


Cardinality Ratio 1: N for $E_1 : E_2$ in R



Structural Constraint (min, max)
on Participation of E in R

COMPANY database - Using Tool to draw ERD



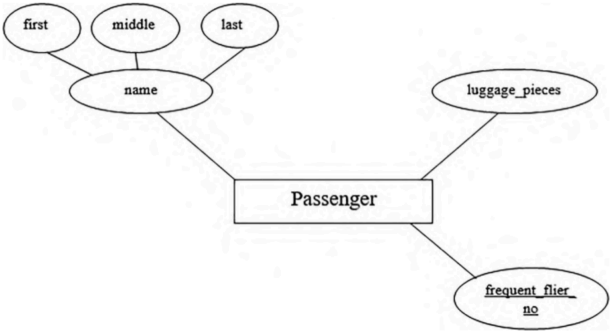
ER Design Methodology

- **Step 1**. Select one primary entity from the database requirements description; Show attributes.
- **Step 2**. Find out if information about attributes refer to another thing.
- **Step 3**. If another entity is appropriate, draw the second entity. Repeat step 2.
- **Step 4**. Connect entities with relationships if relationships exist.
- **Step 5**. Define Cardinality ratio of relationship.
- **Step 6**. Show some sample data.

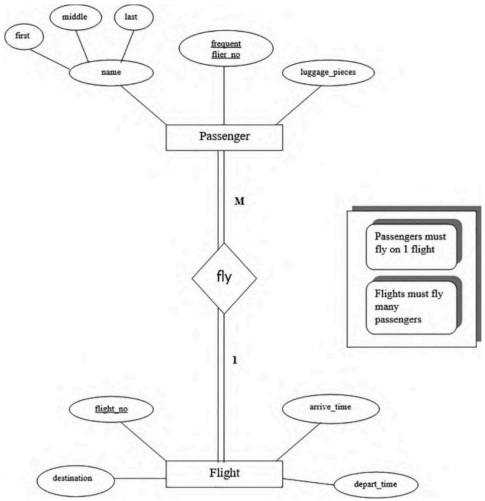
Example

Consider a model for a simplified airport where **PASSENGERS** and **FLIGHTS** are to be recorded. Suppose the attributes of **PASSENGERS** are ***name, luggage_pieces*** and ***frequent_flier_no***. Suppose the attributes for **FLIGHT** are ***flight_no***, ***destination***, ***arrive_time*** and ***depart_time***. Draw the ER diagram.

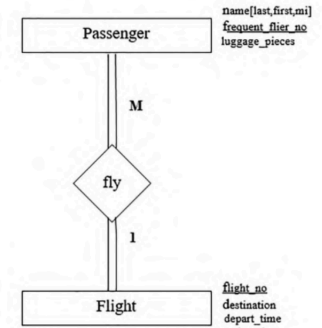
ER Design Methodology:

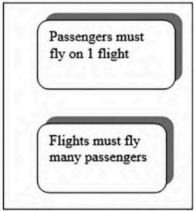


Example


middle
last
first
name
frequent flier_no
luggage_pieces
Passenger
M
fly
1
Passengers must fly on 1 flight
Flights must fly many passengers
flight_no
arrive_time
destination
Flight
depart_time

Example





Some sample data:

Passenger (name, FFN, luggage, flight_no)

name	FFN	luggage	flight_no
Jones, J.J.	12345	3	207
Smith, S.S.	22334	1	101

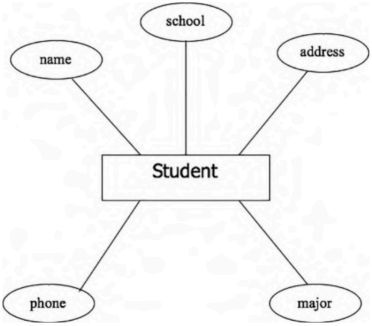
Note: FFN is frequent_flier_no

Flight(flight_no, destination, depart_time, arrive_time)

flight_no	destination	depart	arrive
101	NYC	08:05	10:10
207	Philadelphia	09:34	11:35

Mapping Entity to Relation

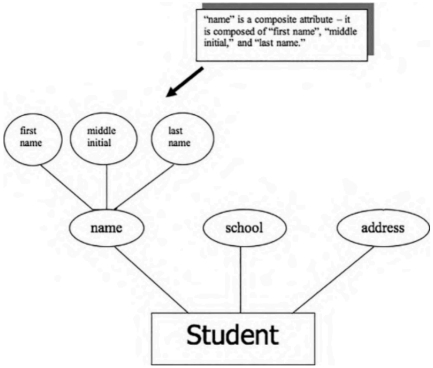
Mapping rule 1: *Mapping strong entities to a relation with attibutes and the primary key. If more than one candidate key is indicated on the ER diagram, choose one for the primary key.*



STUDENT(<u>name</u>, phone, school, address, major)

Mapping Entity to Relation

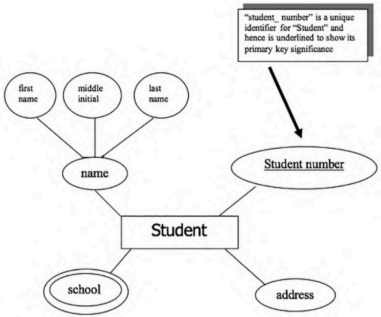
Mapping rule 2: *Mapping composite attributes.*



STUDENT(name.first, name.mid, name.last, school, address)

Mapping Entity to Relation

Mapping rule 3: *Mapping multivalued attributes.*

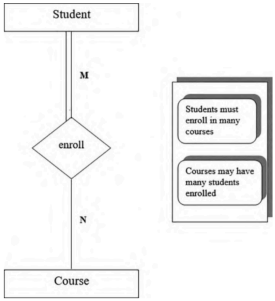


STUDENT(student_number, name.first, name.mid, name.last, address)

STUDENT_SCHOOL(student_number, school)

Mapping Relationship to Relation

Mapping rule 4: *Mapping binary M:N relationships.*



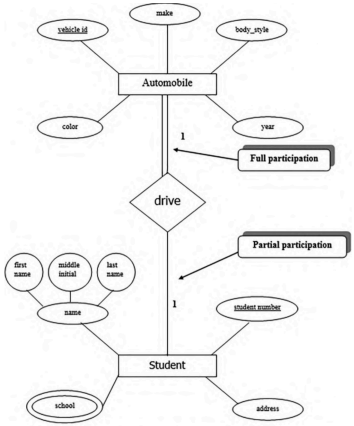
STUDENT(student_number, name.first, name.mid, name.last, address)
COURSE(c_number, cname, credit_hrs)
ENROLL(student_number, c_number)

Mapping Relationship to Relation

Mapping rule 5: *Mapping a binary 1:1 relationship when one side of the relationship has full participation and the other has partial participation.*

AUTOMOBILE(vehicle_id, make, body_style, color, year, student_number)

STUDENT(student_number, name.first, name.mid, name.last, address)



Mapping Relationship to Relation

Mapping rule 6: *Mapping a binary 1:1 relationship when both sides have partial participation constraints.*

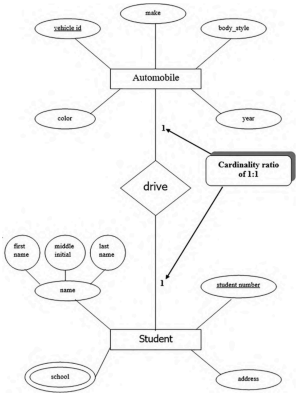
AUTOMOBILE(vehicle_id, make, body_style, color, year)

STUDENT(student_number, name.first, name.mid, name.last, address, vehicle_id)

AUTOMOBILE(vehicle_id, make, body_style, color, year)

STUDENT(student_number, name.first, name.mid, name.last, address)

STUDENT_AUTOMOBILE(student_number, vehicle_id)



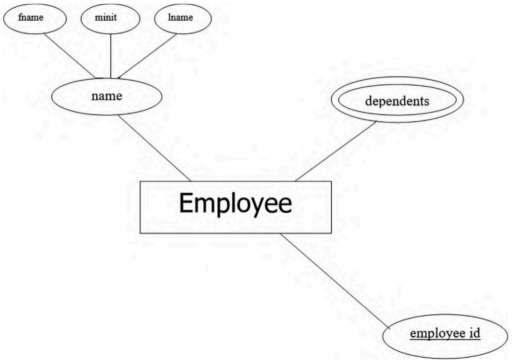
Mapping Relationship to Relation

Mapping rule 7: *Mapping a binary 1:1 relationship when both sides have full participation constraints. Using M:N*

Mapping rule 8: *Mapping binary 1:N relationships when the N side has full participation. Include the key of the entity on the 1 side of the relationship as a foreign key on the N side. Using rule 5.*

Strong Entity

e.g.



Strong Entity

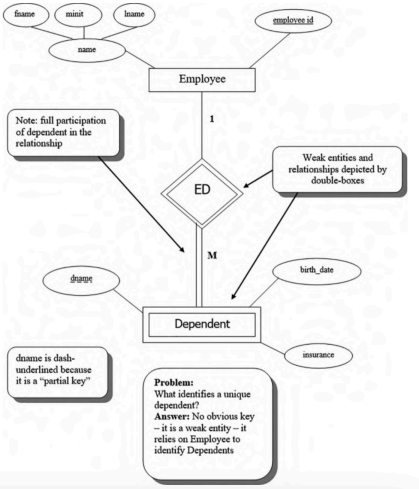
Sample data:

fname	minit	lname	emp ID	dependents
John	J	Jones	0001	John, Jr; Dumas; Sally
Sam	S	Smith	0004	Brenda; Richard
Adam	A	Adams	0007	John; Quincy; Maude
Santosh	C	Saha	0009	Ranu; Pradeep; Mala

Weak Entity

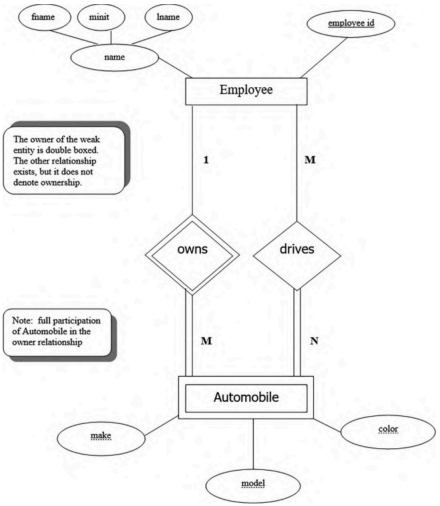
A weak entity clearly will be an entity but will depend on another entity for its existence.
A weak entity will not necessarily have a unique identifer.

- The attribute ***dname*** is a **Partial Key**.
- Underlined ***dname*** with dashes.



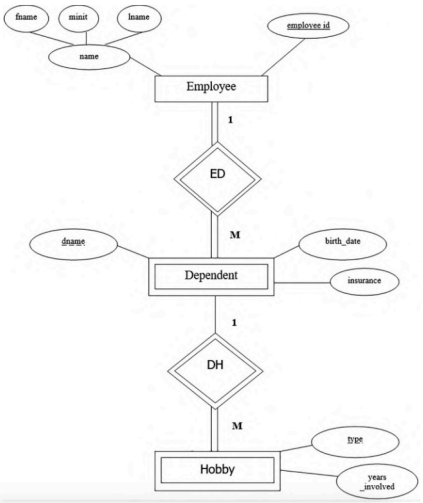
Weak Entity

- The automobile is identifed by the owner.
- Double diamond on the owns relationship and the full participation of the **AUTOMOBILE** entity in the owns relationship.
- All automobiles are driven by some employee.



Weak Entity - Weak Entity

- Entity **EMPLOYEE** is the owner of the weak entity **DEPENDENT**
- Weak Entity **DEPENDENT** is the owner of the weak entity **HOBBY**



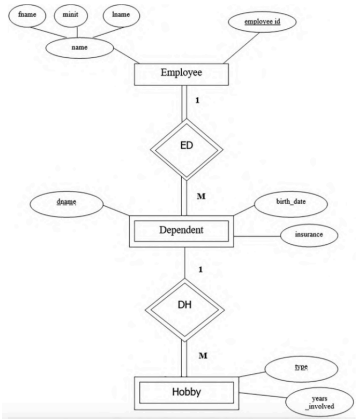
Mapping Weak Entity to Relational DB

Mapping Rule 9: *Mapping weak entities.*

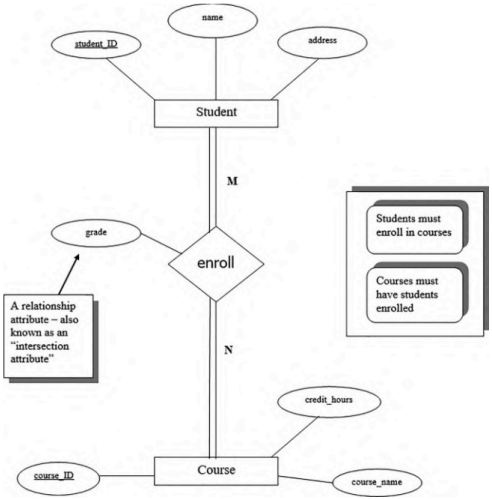
EMPLOYEE(employee_id, fname, minit, lname)

DEPENDENT(employee_id, dname, insurance, birth_date)

HOBBY(employee_id, dname, type, year_involved)



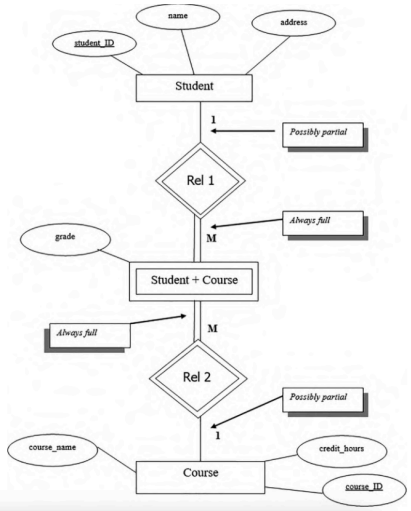
Attributes of Relationship



Change M:N Relationship

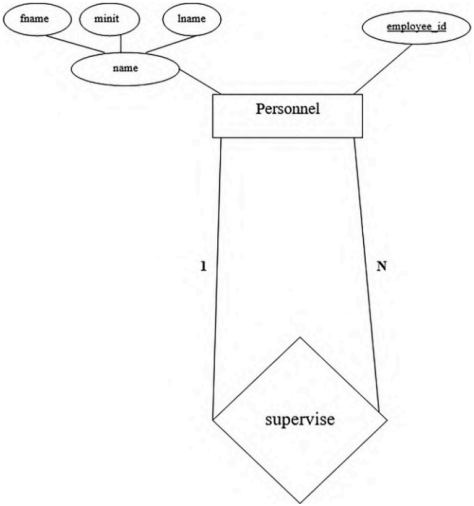
Change to:

- Intersection Weak Entity: STUDENT-COURSE
  - Attribute: grade
  - Key: student_ID + course_ID
- Relationship 1: Rel1
- Relationship 2: Rel2



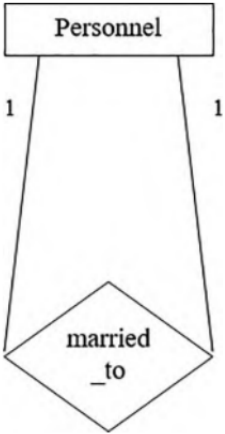
Recursive Relationship

- An employee may supervise many other employees
- Many employees having one supervisor



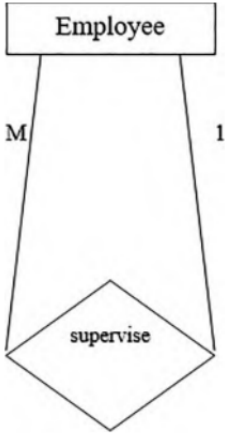
One to One Recursive Relationship

e.g.



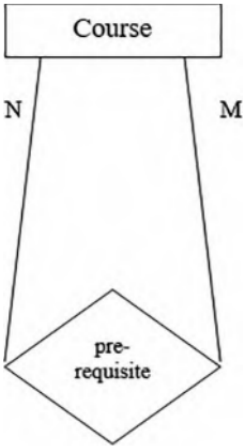
One to Many Recursive Relationship

e.g.



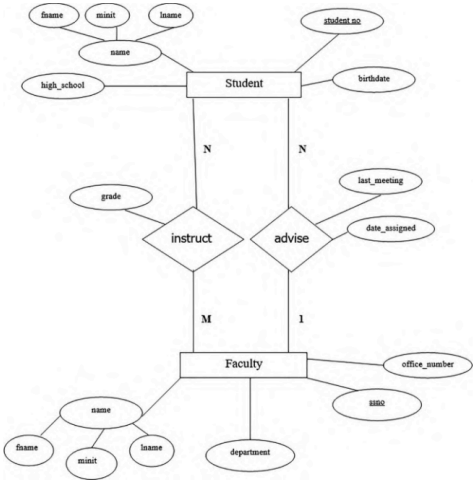
Many to Many Recursive Relationship

e.g.



Multiple Relationship

e.g.



Mapping Recursive Relationship

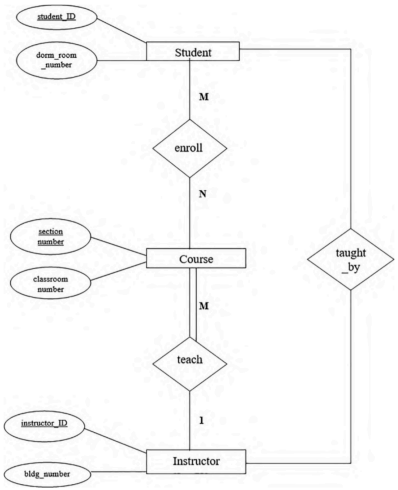
Mapping Rule 10: *Mapping 1:N recursive relationships. Re-include the primary key of the table with the recursive relationship in the same table, giving it some other role name.*

e.g.

PERSONNELL(emplye_id, fname, mname, lname, super_id)

Derived or Redundant Relationship

- Students take courses
- Each course is taught by an instructor
- We do not need a **taught_by** relationship



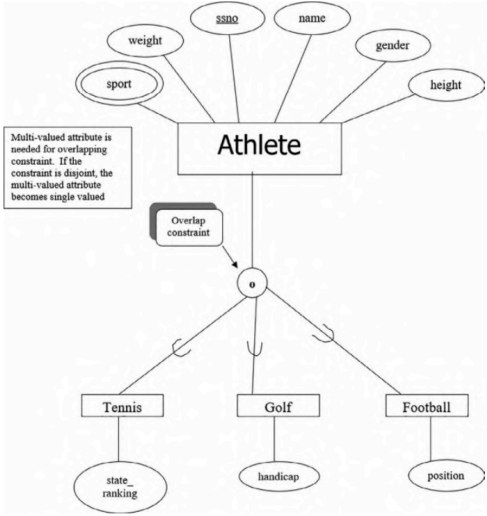
Generalization or Specialization

- Generalization: Super Class
- Specialization: Sub Class

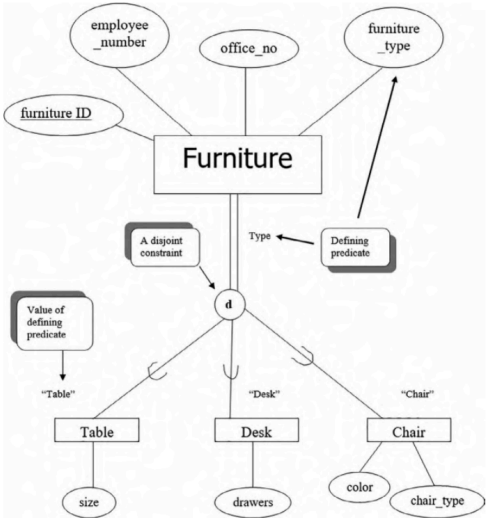
Constraints: There are two types of constraints on the "Sub-class" relationship.

- Overlapping: Superclass entity may contain more than one subclass
- Disjoint: Superclass entity may be a member of only one of the subclasses

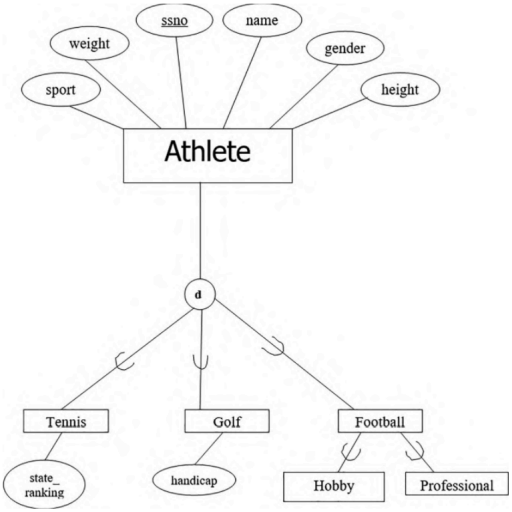
Generalization or Specialization



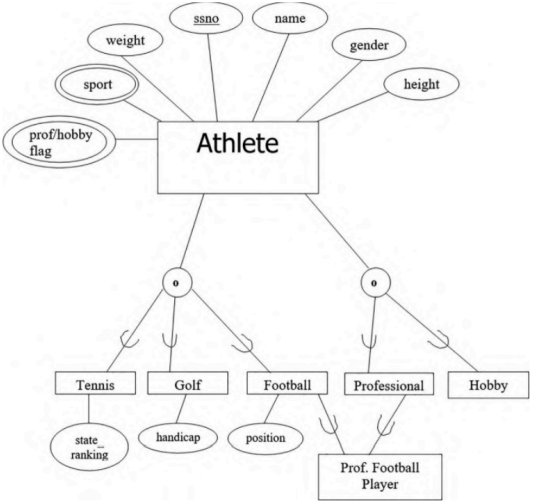
Generalization or Specialization



Subclasses of Subclasses



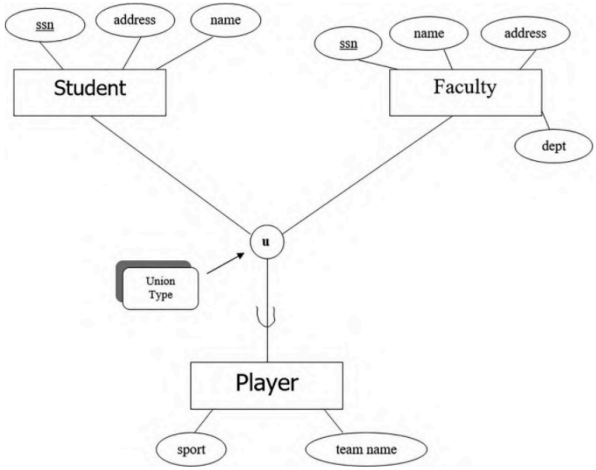
Subclasses of Subclasses



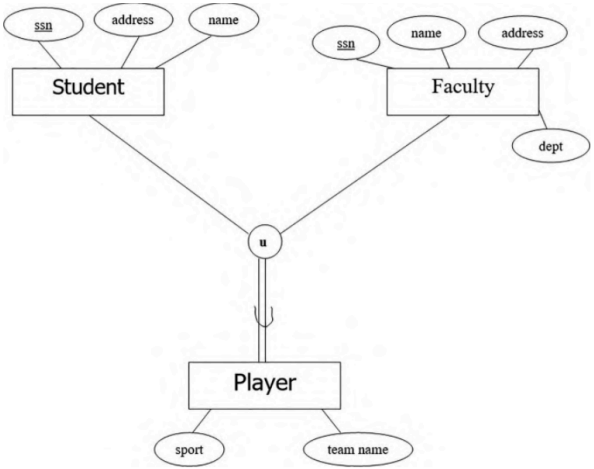
Mapping rules to Relation

- Generalization: New Relation
- Specialization: New Relation with Primary key of Generalization

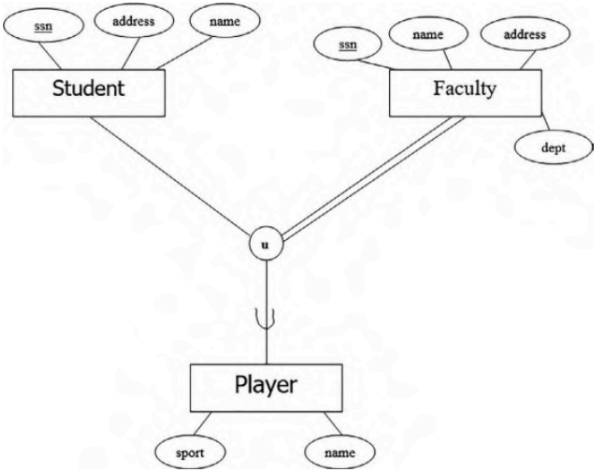
Union Types


ssn
address
name
Student
ssn
name
address
Faculty
dept
u
Union Type
Player
sport
team name

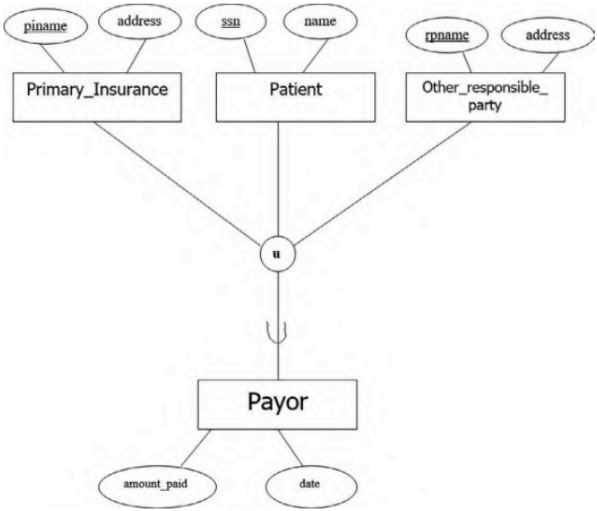
Union Types


ssn
address
name
Student
ssn
name
address
Faculty
dept
u
Player
sport
team name

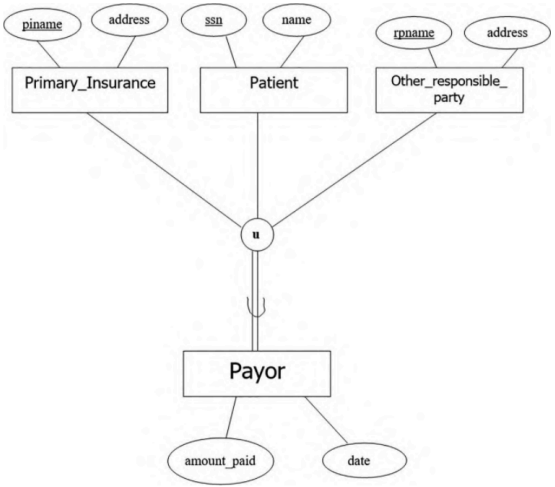
Union Types


ssn
address
name
Student
ssn
name
address
Faculty
dept
u
Player
sport
name

Union Types


piname
address
ssn
name
rpname
address
Primary_Insurance
Patient
Other_responsible_
party
u
Payor
amount_paid
date

Union Types


piname
address
ssn
name
rpname
address
Primary_Insurance
Patient
Other_responsible_
party
u
Payor
amount_paid
date

Mapping rules to Relation

Generalizations have the same primary keys

- Generalizations: New Relations
- Specialization: New Relation with the same as the Primary key of Generalization

Generalizations have diferent primary keys

- Generalizations: New Relations with foreign key to be PK of specialization
- Specialization: New Relation with the same as the Primary key of Generalization

Functional Dependencies Definition

Let:

- $R(A_1, \cdots, A_n)$ be a relational scheme,
- X and Y be subsets of $\{A_1, \cdots, A_n\}$

We say:

- $X \rightarrow Y$, read "*X functionally determines Y*" or "*Y functionally depends on X*" if:

  $\forall (t_1, t_2) \in \forall r(R)$:

  Not possible: $t_1(X) = t_2(X)$ *but* $t_1(Y) \neq t_2(Y)$

- In the FD $X \rightarrow Y$, X is called the *Left side* and Y called the *Right side*.

Notational Conventions

- $A, B, C, \cdots$ stand for single attributes
- $U, V, \cdots, Z$ stand for sets of attributes
- R is used to denote a *relation scheme*
- $r(R)$ *relation*, the current instance of scheme R
- $A_1 \cdots A_n$ set of attributes $\{A_1, \cdots, A_n\}$
- XY stand for $X \cup Y$
- XA or AX stand for $X \cup \{A\}$

e.g.

StudentScore(StudentID, StudentName, SubjectID, SubjectName, Score)

One current **StudentScore**:

StudentScore	(StudentID	StudentName	SubjectID	SubjectName	Score)
	T01	Nam	S01	Database	7
	T01	Nam	S02	Math	6
	T02	Tuan	S01	Database	9
	T02	Tuan	S02	Math	5
	T03	Cuong	S01	Database	8
	T04	Quoc	S02	Math	8

StudentID $\rightarrow$ StudentName
SubjectID $\rightarrow$ SubjectName
StudentID, SubjectID $\rightarrow$ Score

Satisfaction of Dependencies

- r *satisfies* FD $X \rightarrow Y$

  if $\forall (t_1, t_2) \in r(R) : t_1(X) = t_2(X)$ *then* $t_1(Y) = t_2(Y)$

- "if ... then" statement, it can be satisfied either by:
  1. $t_1(X) \neq t_2(X)$ or
  2. $t_1(Y) = t_2(Y)$
- If r does not satisfy $X \rightarrow Y$, the r violate $X \rightarrow Y$.
- If we declared $X \rightarrow Y$ hold for R, then all instance r of scheme R will satisfy $X \rightarrow Y$.
- FD $X \rightarrow \varnothing$ is *trivially satisfied* by any relation.
- FD $\varnothing \rightarrow Y$ is satisfied by those relations in which every tuple has the same $Y-$ *value*.

FDs is an Integrity Constraints

Let: $R(ABC)$; FD: $A \rightarrow B$
There is an *integrity constraint* of scheme R.

- Context: R
- Condition:
  $\forall (t_1, t_2) \in \forall r(R) : t_1.A = t_2.A$

  $t_1.B = t_2.B$

  end.
- Influence table:

	Insert	Delete	Update
R	+	-	+(A/B)

PostgreSQL: create table

```
CREATE TABLE R (A int, B int, C int);
INSERT INTO R VALUES
(1,1,1),
(1,1,2),
(2,2,3);
```

Create a function, it returns trigger type:

```
create or replace function insert_R_fn()
  returns trigger as $$
begin
  if exists (select 1
                from R
                where A = NEW.A
                  and B <> NEW.B) then
            raise exception 'FD: Insert violation';
  end if;
  return NEW;
end;
$$ language plpgsql;
```

Create a trigger, it executes function:

```
create or replace trigger insert_R_trigg
    before insert on R
    for each row
    execute function insert_R_fn();
```

Create a function for update of A:

```
create or replace function update_R_A_fn()
  returns trigger as $$
begin
  if exists (select 1
               from R
               where A = NEW.A
                 and B <> NEW.B) then
          raise exception 'FD: Update A violation';
  end if;
  return NEW;
end;
$$ language plpgsql;
```

Create a trigger for update of A:

```
create or replace trigger update_R_A_trigg
    after update of A on R
    for each row
    execute function update_R_A_fn();
```

Create a function for update of B:

```
create or replace function update_R_B_fn()
  returns trigger as $$
begin
  if exists (select 1
               from R
               where A = NEW.A
                 and B <> NEW.B) then
          raise exception 'FD: Update B violation';
  end if;
  return NEW;
end;
$$ language plpgsql;
```

Create a trigger for update of B:

```
create or replace trigger update_R_B_trigg
    after update of B on R
    for each row
    execute function update_R_B_fn();
```

Inference FD

Let R; A, B, C : attributes.
FD: $A \rightarrow B$ and $B \rightarrow C$ are known to hold in R.
We claim that $A \rightarrow C$ must also hold in R

Proof by Contradiction

Suppose r satisfies $A \rightarrow B$ and $B \rightarrow C$,

but $\exists (t_1, t_2) \in r : t_1.A = t_2.A$ *and* $t_1.C \neq t_2.C$

- if $t_1.B \neq t_2.B$, then r *violate* $A \rightarrow B$
- if $t_1.B = t_2.B$ but $t_1.C \neq t_2.C$ then r *violate* $B \rightarrow C$

Hence r must satisfy $A \rightarrow C$ ■

Direct Proof

$\forall (t_1, t_2) \in \forall r(R) \ : \ t_1.A = t_2.A \implies t_1.B = t_2.B \quad (A \rightarrow B)$
$\implies t_1.C = t_2.C \quad (B \rightarrow C)$

from FD definition $\implies A \rightarrow C$ ■

Logically Inference

– Let F set of functional dependencies hold on R
– $X \rightarrow Y$ be a functional dependency.

- F *logically inference* $X \rightarrow Y$ written $F \models X \rightarrow Y$ if $\forall r(R)$ *satisfies* F *also satisfies* $X \rightarrow Y$

e.g. $F = \{ A \rightarrow B,\ B \rightarrow C \}$
then $A \rightarrow C$ is logically inference by F
that is, $\{ A \rightarrow B,\ B \rightarrow C \} \models A \rightarrow C$

Amstrong's Inference Axioms

Giving U be a *universal set* of attributes. F be a FDs set involving only attributes in U. The inference rule are:

1. *Reflexivity*. If $Y \subseteq X \subseteq U$, then $X \to Y$ is logically implied by F. This rules gives the *trivial dependencies* . It hold in every relation.
2. *Augmentation*. $\{ X \to Y,\ Z \subseteq U \} \models XZ \to YZ$.
3. *Transitivity*. $\{ X \to Y,\ Y \to Z \} \models X \to Z$.

e.g.

Let $R(ABCD)$, $F = \{A \to C, B \to D\}$.

$F \models AB \to ABCD$?

1. $A \to C$ (given)
2. $AB \to ABC$ [augmentation of (1) by AB]
3. $B \to D$ (given)
4. $ABC \to ABCD$ [augmentation of (3) by ABC]
5. $AB \to ABCD$ [transitivity applied to (2) and (4)] ■

Lemma

Amstrong's Axioms are sound. That is,

if $X \rightarrow Y$ is deduced from F using the axioms, then $X \rightarrow Y$ is true in any relation in which the dependencies of F are true.

Proof:

Reflexivity. Clearly sound.

Augmentation. Suppose relation r satisfies $X \rightarrow Y$, but

$\exists (t_1, t_2) : t_1(XZ) = t_2(XZ)$ *and* $t_1(YZ) \neq t_2(YZ)$

$\Longrightarrow \ t_1.Y \neq t_2.Y$

So $t_1.X = t_2.X$ *and* $t_1.Y \neq t_2.Y$, violating $X \rightarrow Y$ holds for r. ■

Transitivity. Similarly, as an exercise.

Lemma

There are several other inference *rules* that follow from Amstrong's Axioms
Lemma:

1. *Union*. $\{ X \to Y,\ X \to Z \} \models X \to YZ$
2. *Pseudotransitivity*. $\{ X \to Y,\ WY \to Z \} \models WX \to Z$
3. *Decomposition*. $\{ X \to Y,\ Z \subseteq Y \} \models X \to Z$

Proof

Proof:

Union. Given $X \to Y \models X \to XY$ (augment)

Given $X \to Z \models XY \to YZ$ (augment)

By transitivity $\{ X \to XY,\ XY \to YZ \} \models X \to YZ$

Pseudotransitivity. Given $X \to Y \models WX \to WY$ (augment)

Since given $WY \to Z \models WX \to Z$ (transitivity)

Decomposition. Given $Z \subseteq Y \models Y \to Z$ (reflexivity)

Given $X \to Y$, by transitivity, $\models X \to Z$ ■

e.g.

$F = \{ AB \to CD,\ C \to B,\ A \to C,\ BD \to G,\ H \to K \}$

Show $F \models AH \to GK$

$AB \to CD \models AB \to D$ [decomposition] (1)

$\{ A \to C,\ C \to B \} \models A \to B$ [transitivity] (2)

(2) $A \to B \models A \to AB$ [augment] (3)

$\{$ (3) $A \to AB$, (1) $AB \to D \} \models A \to D$ [transitivity] (4)

$\{$ (2) $A \to B$, (4) $A \to D \} \models A \to BD$ [union] (5)

$\{$ (5) $A \to BD,\ BD \to G \} \models A \to G$ [transitivity] (6)

$H \to K \models GH \to GK$ [augment] (7)

$\{$ (6) $A \to G,\ GH \to GK \} \models AH \to GK$ [pseudotransitivity] ■

Closure of FDs Set

- We define F^+, the *closure* of F, to be the set of functional dependencies that are logically implied by F; i.e.,
- $F^+ = \{ X \to Y \mid F \models X \to Y \}$

e.g. $F = \{ AB \to C,\ C \to B \}$ hold on $R(ABC)$, then:

$F^+ = \{ A \to A,\ AB \to A,\ AC \to A,\ ABC \to A,\ B \to B,$

$AB \to B,\ BC \to B,\ ABC \to B,\ C \to C,\ AC \to C,$

$BC \to C,\ ABC \to C,\ AB \to AB,\ ABC \to AB,\ AC \to AC,$

$ABC \to AC,\ BC \to BC,\ ABC \to BC,\ ABC \to ABC,$

$AB \to C,\ AB \to AC,\ AB \to BC,\ AB \to ABC,\ C \to B,$

$C \to BC,\ AC \to B,\ AC \to AB \}$

e.g.

Let $R(ABC)$ and $F = \{ A \to B,\ B \to C \}$. Then F^+ consists of all those dependencies $X \to Y$ such that either

1. X contains A e.g., $ABC \to AB$, $AB \to BC$, or $A \to C$,
2. X contains B but not A, and Y does not contain A e.g., $BC \to B$, $B \to C$, or $B \to \varnothing$, and
3. $X \to Y$ is one of the three dependencies $C \to C$, $C \to \varnothing$, or $\varnothing \to \varnothing$.

Closure of Attribute Set

Let:

- R be an attributes set.
- F be a functional dependencies set hold on R.
- X be a subset of R.

We define X_F^+ is the *closure* of X (with *respect* of F) is the set of attributes A such that $X \to A$ can be logically implied from F (by FD definition).

$$X_F^+ = \{ A \mid F \models X \to A \}$$

e.g.

$R(ABCD)$, $F = \{ B \to D,\ A \to B,\ C \to B \}$

$A_F^+ = ABD$

Algorithm

Closure of a set of attributes with respect to a set of functional dependencies.

Input: R, F, $X \subseteq R$

Output: X_F^+

Method: Computing a sequence attributes sets $X_F^{(0)}, X_F^{(1)}, \ldots$ by the rule:

1. $X_F^{(0)} \leftarrow X$
2. Repeat $Y \rightarrow Z \in F : Y \subseteq X_F^{(i)}$

   $X_F^{(i+1)} \leftarrow X_F^{(i)} \cup Z$

   Until $X_F^{(i+1)}$ is not change

Since $X = X_F^{(0)} \subseteq \cdots \subseteq X_F^{(i)} \subseteq \cdots \subseteq R$, and R is finite, we must eventually reach i such that $X_F^{(i)} = X_F^{(i+1)}$.

e.g.

$$F = \{\ AB \to C, \quad D \to EG, \quad C \to A, \quad BE \to C$$
$$BC \to D, \quad CG \to BD, \quad ACD \to B, \quad CE \to AG\}$$

$X = BD$, compute X_F^+:

$X_F^{(0)} = BD$
$X_F^{(1)} = BDEG$
$X_F^{(2)} = BCDEG$
$X_F^{(3)} = ABCDEG$
$X_F^{(3)} = X_F^{(4)} = \cdots$

Thus $(DB)_F^+ = ABCDEG$

Lemma

Dependencies Logically Implied by Closure Set.

Let R , F , $X \subseteq R$, X_F^+, then:

- $X \rightarrow X_F^+$
- $Z \subseteq X_F^+$, then $X \rightarrow Z$

Proof: $X \rightarrow Z$:

Suppose $\exists (t_1, t_2) : t_1.X = t_2.X$, but $t_1.Z \neq t_2.Z$

So $\exists\ A \in Z : t_1.A \neq t_2.A$. That is contradiction with closure set definition, because of $A \in Z \in X_F^+$ ■

Similarly with $X \rightarrow X_F^+$ ■

Testing Membership in F^+

Let $X \rightarrow Y$ and F
Determine $F \models X \rightarrow Y$? We have three methods:

1. Using FD definition
2. Using Amstrong's axioms
3. Test if $X \rightarrow Y \in F^+$

In the 3th method, F^+ can be very lager

How to show $X \rightarrow Y \in F^+$ without generating all of F^+

Algorithm:

1. X_F^+
2. *if* $Y \subseteq X_F^+$ *then* $F \models X \rightarrow Y$
   if $Y \not\subseteq X_F^+$ *then* $F \not\models X \rightarrow Y$

e.g.

$F = \{\ A \to D,\ AB \to E,\ BI \to E,\ CD \to I,\ E \to C\ \}$

Proofing items below by using Amstrong's axioms as an exercise.

1. $AE \to AI \in F^+$?

   $AE_F^+ = ACDEI$ contain the right side of the FD, so

   $AE \longrightarrow AI \in F^+$ or $F \models AE \to AI$ ■

2. $DB \to C \in F^+$?

   $DB_F^+ = DB$ not contain the right side of the FD, so

   $DB \to C \notin F^+$ or $F \not\models DB \to C$ ■

Covers and Equivalence of Dependencies Sets

Definition:

- Two FDs F and G over R are *equivalent* if $F^+ = G^+$
- Denote $F \equiv G$
- If $F \equiv G$, then F is a *cover* for G

Test whether F and G are equivalent:

- *Step 1:* $F \models G$
  - Test every $g : X \to Y \in G$ is $g \in F^+$
  - $g : X \to Y \in G$, compute X_F^+
  - If $Y \subseteq X_F^+$, then $g \in F^+$, if not: $g \notin F^+$
  - If $\exists\, g \in G, \notin F^+$, then surely $F^+ \neq G^+$
- *Step 2:* $G \models F$
  Analogous manner *Step 1:*, test every $f \in F$ is $f \in G^+$

e.g.

$F = \{\ A \to BC,\ A \to D,\ CD \to E\ \}$
$G = \{\ A \to BCE,\ A \to ABD,\ CD \to E\ \}$
$F \equiv G$?

Step 1: $F \models G$.
$A \to BCE$: $A_F^+ = ABCDE \Rightarrow F \models A \to BCE$
$A \to ABD$: $A_F^+ = ABCDE \Rightarrow F \models A \to ABD$
$CD \to E$: $CD_F^+ = CDE \Rightarrow F \models CD \to E$

Step 2: $G \models F$.
$A \to BC$: $A_G^+ = ABCED \Rightarrow G \models A \to BC$
$A \to D$: $A_G^+ = ABCED \Rightarrow G \models A \to D$
$CD \to E$: $CD_G^+ = CDE \Rightarrow G \models CD \to E$

$F \equiv G$ ■

Minimum Covers

Definition

- F have many cover sets, its equivalent to F
- We define a set F_c is a *minimal cover* of F if:

1. Every *right side* of FD in F_c is a single attribute
2. $\nexists\, X \to A \in F_c \ : \ \{\ F_c - \{\ X \to A\ \}\ \} \equiv F_c$
3. $\nexists\, X \to A \in F_c$ *and* $Z \subseteq X$:
   $\{\ F_c - \{X \to A\}\ \} \cup \{Z \to A\} \equiv F_c$

One dependencies G maybe there are many minimum covers.

Algorithm

Find one minimum cover of dependencies set.
Input: Dependencies F.
Output: One minimum cover.
Method: Compute a sequence steps:

1. Decompose right side FDs in F to one attribute.
2. For each $X \to A$ in F:
   if $F - \{X \to A\} \equiv F$ then
   $F = F - \{X \to A\}$
3. For each $X \to A$ in F:
   if $Z \subset X$ and $\left\{ F - \{X \to A\} \right\} \cup \{Z \to A\} \equiv F$ then
   $F = \left\{ F - \{X \to A\} \right\} \cup \{Z \to A\}$.

Easy proofing three steps, we see as an exercise ■

e.g.

$$F = \{\ AB \to C,\quad D \to EG,\quad C \to A,\quad BE \to C,\quad BC \to D$$
$$CG \to BD,\quad ACD \to B,\quad CE \to AG\ \}$$

Step 1, split right side FDs:

$$AB \to C,\quad D \to E,\quad D \to G,\quad C \to A,\quad BE \to C,\quad BC \to D$$
$$CG \to B,\quad CG \to D,\quad ACD \to B,\quad CE \to A,\quad CE \to G$$

Step 2, remove $CE \to A$, $CG \to B$

Step 3, $ACD \to B$ replaced by $CD \to B$

One minimum cover

$$F_c = \{\ AB \to C,\quad D \to E,\quad D \to G,\quad C \to A,\quad BE \to C,$$
$$BC \to D,\quad CG \to D,\quad CD \to B,\quad CE \to G\ \}$$

Projected Functional Dependencies

R, F ; R_1: Attributes Set.

F_1 = *projected FDs of* F *on* R_1,

denote $F_1 = \pi_{R_1}(F)$:

- F_1 follow from F and
- Involve only attributes of R_1.

Algorithm

Input: R, F ; R_1 : Attributes Set

Output: $F_1 = \pi_{R_1}(F)$

Method: Compute a sequence steps

1. $F_1 \leftarrow \varnothing$
2. *For each* $X \subset \mathcal{R}_1$
   - Compute X_F^+
   - If $X_F^+ \cap R_1 \neq \varnothing$:

     $$F_1 = F_1 + \{\ X \rightarrow X_F^+ \cap R_1\ \}$$
3. $F_1 \leftarrow$ a minimal cover of F_1

e.g.

$R(ABCD)$,$F = \{\ A \to B,\ B \to C,\ C \to D\ \}$

Find $F_1 = \pi_{R_1(ACD)}(F)$

- *First*, $A_F^+ = ABCD \Rightarrow F_1 = \{\ A \to C,\ A \to D\ \}$
- *Next*, $C_F^+ = CD \Rightarrow F_1 = \{\ A \to C,\ A \to D,\ C \to D\ \}$
- *Since* $A_F^+ = R_1$, no any superset of A can find new FDs.

  Thus, the only CD can be compute, but $CD^+ = CD$, add nothing.
- *Done* $F_1 = \{A \to C,\ A \to D,\ C \to D\}$
- Find a minimum cover of F_1,

  Thus, $F_1 = \{\ A \to C,\ C \to D\ \}$ ■

Keys by Functional Dependencies

$R(A_1, \cdots, A_n)$ and F; subset $K \subseteq A_1 A_2 \cdots A_n$

K is a *key* of R if:

1. $K \to A_1 A_2 \cdots A_n$ is in F^+, and
2. $\nexists\ Y \subsetneq K:\ Y \to A_1 A_2 \cdots A_n$ in F^+

- May be more than one key, one call *primary key*
- The keys not chose primary key is called *candidate key*
- Set consists key is called *super key*
- One key show in relation scheme by *an underline*

e.g. $R(ABC),\ F = \{\ A \to B,\ B \to C\ \}$.
Only one key A, since $A \to ABC$ is in F^+ ($F \models A \to ABC$),
and *no set X* dose not contain A, *is also $X \to ABC$*.

$\mathcal{R}(\underline{A}BC)$

e.g.

– $R_1(ABCD)$, $F_1 = \{\ AB \to CD,\ C \to A,\ D \to BC\ \}$

R_1 have two keys: AB and D

$R_1(\underline{AB}\ C\ \underline{D})$

– $R_2(ABC)$, $F_2 = \emptyset$

R_2 have one key, it all attributes.

$R_2(\underline{ABC})$

– $R_3(ABC)$, $F_3 = \{\ A \to B,\ B \to C,\ C \to A\ \}$

R_3 have three keys: A, B and C

$R_3(\underline{A}\ \underline{B}\ \underline{C})$

Algorithm

Input: R , F.
Output: All keys of R.
Method: Computing a sequence steps:

1. Generate all subsets of R, not $\varnothing$ set. *Sorting it by ascending*.
2. Computing closure those sets.
3. Only keep the sets, such that *its closure equal* R.
4. Remove the sets, it *contains any other sets* in list.
5. All sets remain are the all keys of relation scheme R.

e.g.

$R(ABC)$, $F = \{\ A \to B,\ B \to C,\ B \to A\ \}$
Find all keys.

Step 1, Step 2:
$A^+ = ABC, \quad B^+ = ABC, \quad C^+ = C, \quad AB^+ = ABC,$
$AC^+ = ABC, \quad BC^+ = ABC, \quad ABC^+ = ABC$
Step 3:
$A^+ = ABC, \quad B^+ = ABC, \quad AB^+ = ABC$
$AC^+ = ABC, \quad BC^+ = ABC, \quad ABC^+ = ABC$
Step 4:
A
B
Step 5:
$\mathcal{R}(\underline{A}\ \underline{B}\ C)$

Remarkable

We note some thing about keys algorithm above:

1. Attributes do not in FDs, must in every keys.
2. Attributes do only in left side FDs, then it must in every keys.
3. Attributes do only in right side FDs, then it must do not in any key.
4. Attributes do both in left side and right side FDs, then it either in or not in any key.

Algorithm

Input: Q , F.
Output: All keys on Q.
Method: Computing a sequence steps:

1. L : attributes in the *left side* of FDs
   R : attributes in the *right side* of FDs
   $NF = Q - (L + R)$: attributes *do not in* FDs
   $OL = L - R$: attributes *only in left side* of FDs
   $LR = L \cap R$: att. *both in the left side and right side* of FDs
2. Generate all subsets of *LR including* $\varnothing$.
   Sorting it by ascending.
3. Add *NF* and *OL* into subsets above.
4. Computing closure for all subsets.
   Decide the keys like general algorithm.

e.g.

$Q(ABCDEG)$, $F = \{\ B \to CD,\ D \to C,\ C \to EG\ \}$.
Find all keys.

Step 1: $L = BCD, \quad R = CDEG$
$NF = A, \quad OL = B, \quad LR = CD$
Step 2: $\varnothing, \quad C, \quad D, \quad CD$
Step 3: $AB, \quad CAB, \quad DAB, \quad CDAB$
Step 4: $AB^+ = ABCDEG$
~~CAB~~
~~DAB~~
~~$CDAB$~~

One key result: AB

$Q(\underline{AB}CDEG)$

e.g. $Q(ABCD)$, $F = \{\ B \to CD,\ A \to C\ \}$, find all keys.

e.g.

$Q(ABCD)$, $F = \{\ AB \to CD,\ CD \to AB,\ B \to C\ \}$.
Find all keys.

Step 1: $L = ABCD$, $R = ABCD$, $NF = \varnothing$, $OL = \varnothing$, $LR = ABCD$

Step 2: $\varnothing$, D, BC, ABD, A, AB, BD, ACD
B, AC, CD, BCD, C, AD, ABC, $ABCD$

Step 3: Nothing change

Step 4: $\varnothing$, $AB^+ = ABCD$, $CD^+ = CDAB$, ~~ACD~~,
$A^+ = A$, $AC^+ = AC$, ~~ABC~~, ~~BCD~~,
$B^+ = BC$, $AD^+ = AD$, ~~ABD~~, ~~$ABCD$~~,
$C^+ = C$, $BC^+ = BC$,
$D^+ = D$, $BD^+ = BDCA$

Three keys result: $(AB), (BD), (CD)$

FDs logically implied by Keys

R , F,
$K_1, K_2, \cdots K_n$: keys of R, then we have FDs:

$K_1 \rightarrow R$, or $K_1 \rightarrow (R - K_1)$
$\cdots$
$K_n \rightarrow R$, or $K_n \rightarrow (R - K_n)$

e.g.
$R(ABCDE)$, $F = \{\ AB \rightarrow CD,\ D \rightarrow E,\ BC \rightarrow A\ \}$

Two keys: $K_1 = AB$, $K_2 = BC$
Then, there are FDs logically implied by keys:

$AB \rightarrow CDE$
$BC \rightarrow ADE$

Normal Form

- Base on Keys checking system of DBMS, we have a concept to check all FDs on relation.
- This concept is call *Normal Form*.

A good Normal form has mainly two purposes:

- Removing duplicated data, redundancy (repetition) data.
- Ensuring data is logically stored.

First Normal Form (1NF)

- Relation scheme R is in *first normal form (1NF)* if every attribute A in R, Dom(A) are atomic.
- That is, the values in the domain are not lists or sets of values or composite values.
- Database scheme D is in 1NF if every relation scheme in D is in 1NF.

e.g.

Relation **Gender**, shown below, is not in 1NF because it contains values that are sets of atomic values.

Gender(Name, Sex)

Gender	**(Name**	**Sex)**
	{Tuan, Nam, Cuong}	male
	{Nhung, Hang}	female

e.g.

To be in INF, gender should be stored like this:

Gender(Name, Sex)

Gender	**(Name**	**Sex)**
	Tuan	male
	Nam	male
	Cuong	male
	Nhung	female
	Hang	female

Specially *DateTime* data type is a type with atomic domain.

e.g.

Assign(Flight, Day, Pilot, Gate)

with $F = \{\ Flight, Day \rightarrow Pilot;\ Flight \rightarrow Gate\ \}$

Key: $\{Flight, Day\}$

Assign	**(Flight**	**Day**	**Pilot**	**Gate)**
	VN123	20/10/2018	Cuong	7
	VN123	27/10/2018	Nhan	7
	VN456	28/10/2018	Tuan	8

We would like to update the first tuple:

ssign(VN123, 20/10/2018, Cuong; Gate = 9)

e.g.

Assign	(Flight	Day	Pilot	Gate)
	VN123	20/10/2018	Cuong	9
	VN123	27/10/2018	Nhan	7

Assign will violate the FD *Flight* $\rightarrow$ *Gate*.

To avoid violating the FD, every time an update is made, we have to scan the relation and update the gate number every place the flight number appears.

The flight number and gate number information is duplicated, thus making the *data redundant*.

e.g.

We can represent the same information as a database of two relations, **PilotAssign** and **GateAssign**:

PilotAssign

(Flight	Day	Pilot)
VN123	20/10/2018	Cuong
VN123	27/10/2018	Nhan
VN456	28/10/2018	Tuan

GateAssign

(Flight	Gate)
VN123	7
VN456	8

e.g.

To reconstruct the original relation Assign, taking

$$PilotAssign \bowtie GateAssign$$

To update anomaly in Assign, altering to change in GateAssign.

We have also removed data redundancy of Gate attribute.

Fully dependent

- Given R , F;
  X, $Y \subseteq R$
- Y is called *fully dependent* upon X if:
  1. $X \rightarrow Y \in F^+$
  2. $\nexists\, X' \subsetneq X : X' \rightarrow Y \in F^+$

e.g. $F = \{\ Flight, Day \rightarrow Pilot, Gate;\ \ Flight \rightarrow Gate\ \}$

Gate is *not fully dependent* upon $\{Flight, Day\}$.

Pilot is *fully dependent* upon $\{Flight, Day\}$.

Prime and nonprime attributes

- Given R , F
- A is an attribute in R
- A is *prime* in R with respect to F if A is *contained in some key* of R.
- Otherwise A is *nonprime* in R.

e.g. **Assign(Flight, Day, Pilot, Gate)**

with $\mathcal{F} = \{Flight, Day \rightarrow Pilot; Flight \rightarrow Gate\}$

Key: $\{Flight, Day\}$

Prime: *Flight* and *Day*.
Nonprime: *Pilot* and *Gate*.

Second normal form (2NF)

- R is in *second normal form (2NF)* respect to FDs F
- if it is in 1NF and every nonprime attribute is fully dependent on every key of R.
- A database scheme D is in second normal form if every relation scheme in 2NF.

e.g.

Assign(Flight, Day, Pilot, Gate)

with $F = \{\ Flight, Day \rightarrow Pilot;\ Flight \rightarrow Gate\ \}$

Key: $\{Flight, Day\}$

Assign is not in 2NF, because *Gate* is not fully dependent on key $\{Flight, Day\}$.

If we let $D = \{\ R_1(Flight, Day, Pilot);\ R_2(Flight, Gate)\ \}$,
then both R_1 and R_2 are in 2NF,

so D is in 2NF.

Transitively Dependent

- R , F
- $X \subseteq R$, $A \in R$
- A : *transitively dependent* upon X in R if there is a $Y \subseteq R$ with:
  - $X \rightarrow Y \in F^+$
  - $Y \nrightarrow X \in F^+$
  - $Y \rightarrow A \in F^+$
  - $A \notin XY$

e.g.

Conference(Room, Day, Reporter, Name)

$F = \{\ Room, Day \rightarrow Reporter, Name;\ \ Reporter \rightarrow Name\ \}$

- $Room, Day \rightarrow Reporter$
- $Reporter \nrightarrow Room, Day$
- $Reporter \rightarrow Name$
- $Name \notin \{Room, Day\} \cup \{Reporter\}$

Name is *transitively dependent*

Third Normal Form (3NF)

Classic Definition

R is in *third normal form (3NF)* if:

- It is in 1NF, and
- *no nonprime attribute* in R is *transitively dependent* upon a key of R.

Database scheme D is in *third normal form* if every relation scheme R in D is in third normal form.

e.g.

Conference(Room, Day, Reporter, Name)

$F = \{\ Room, Day \rightarrow Reporter, Name;\ \ Reporter \rightarrow Name\ \}$

It is not in 3NF because of the transitive dependency of *Name* upon $\{Room, Day\}$.

If $D = \{\ R_1(Room, Day, Reporter);\ \ R_2(Reporter, Name)\ \}$, then

D is in 3NF because both R_1 and R_2 are in 3NF.

Third Normal Form (3NF)

Modern Definition

R is in *third normal form (3NF)* if:

- It is in 1NF, and
- Whenever $X \rightarrow Y$ is a nontrivial FD in F^+, either
  - X is a superkey, or
  - Each $B \in Y$ is a member of some key of R

Database scheme D is in *third normal form* if every relation scheme R in D is in third normal form.

Lemma 3NF $\Rightarrow$ 2NF

Any relation scheme R in 3NF is in 2NF.

Proof Using method:

- $(\mathcal{X} \Rightarrow \mathcal{Y}) \Leftrightarrow (\neg\mathcal{Y} \Rightarrow \neg\mathcal{X})$
- $(3NF \Rightarrow 2NF) \Leftrightarrow (\neg 2NF \Rightarrow \neg 3NF)$
- Partial dependency $\Rightarrow$ Transitive dependency.

R is not in 2NF $\Rightarrow$ exists non-prime attribute A is not fully dependent upon key $K \subseteq R$. So that $\exists\ K' \subsetneq K$:

- $K' \rightarrow A \in F^+$ and because of
- $K' \nrightarrow K \in F^+$
- $A \notin K = K' \cup K$

Therefore A is transitively dependent upon key K, so that R is not in 3NF. ■

Boyce-Codd Normal Form (BCNF)

Classic Definition

R : *Boyce-Codd normal form (BCNF)* if:

- It is in 1NF, and
- *No attribute* in R is *transitively dependent* upon a key of R.

Database scheme D is in *Boyce-Codd normal form* if every relation scheme R in D is in Boyce-Codd normal form.

- Because of *(attribute in R)* $\supseteq$ *(nonprime attribute in R)*
- So *if R is in BCNF, then in 3NF*.

Boyce-Codd Normal Form (BCNF)

Modern Definition

R : *Boyce-Codd normal form (BCNF)* if:

- It is in 1NF, and
- for every $Y \subseteq R$ and for every attribute $A \in (R - Y)$
- if $Y \rightarrow A$, then $Y \rightarrow R$
- That is, if Y non-trivially determines any attribute of R, then Y is a superkey for R.

Database scheme D is in *Boyce-Codd normal form* if every relation scheme R in D is in Boyce-Codd normal form.

Overview

Student(Id, Name, Addr, Hobby)

with $F = \{ Id \rightarrow Name, Addr \}$

Key: $\{Id, Hobby\}$

Student is in 1NF, not in 2NF, 3NF or BCNF.

e.g. **Assign(Manager, Project, Branch)**, with F:

$Manager \rightarrow Branch$: Each manager works in a particular branch.

$Project, Branch \rightarrow Manager$: Each project has several managers, and runs on several branches; however, a project has a unique manager for each branch.

Key: $\{Manager, Project\}, \{Project, Branch\}$

Assign is in 3NF, not in BCNF.

Preserve Information from a Decomposition ?

SuppInfo(Name, Addr, Item, Price), with

$F = \{\ Name, Item \rightarrow Price;\ Name \rightarrow Addr\ \}$

SuppInfo is in 1NF. We can replace it into:

$\rho = \{\ Supp(Name, Addr)\ in\ BCNF,$

$SuppDetail(Name, Item, Price)\ in\ BCNF\ \}$

- r is a relation of SuppInfo.
- Instead of store it in SuppInfo, we store it by two relation:
  1. $r(Supp) = \pi_{Name,Addr}(r)$, and
  2. $r(SuppDetail) = \pi_{Name,Item,Price}(r)$

Question: $r(Supp)$ and $r(SuppDetail)$ contain the same data as r.

Only way to recover r is by taking $r(Supp) \bowtie r(SuppDetail)$.

It is right if: $r = r(Supp) \bowtie r(SuppDetail)$

Preserve Information Decomposition

Decomposition of $\{R,\ F\}$ is its replacement:

- $\rho = \{\ R_1, R_2, \cdots, R_k\ \}$
- $R = R_1 \cup R_2 \cup \cdots \cup R_k$
- R_i is no requirement to be disjoin.

This decomposition is *Preserve information* if for every $r(R)$ satisfying F:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \cdots \bowtie \pi_{R_k}(r)$$

In general:

$$r \subseteq \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \cdots \bowtie \pi_{R_k}(r)$$

Tableau Test for Preserve Information

Tableau, *Chase* Algorithm.

Input: $R = \{ A^1, A^2, \cdots, A^n \}$, F,
decomposition $\rho = \{ R_1, R_2 \cdots, R_k \}$

Ouput: Whether ρ is *Recovering information*.

Method:

1. Construct a *tableau* T: n columns, k rows.
   - Column A^j corresponds to attribute A^j
   - Row i corresponds to R_i
   - For all (i, j) cell:

     If $A^j \in R_i$:
     $(i, j) \leftarrow 0$
     else
     $(i, j) \leftarrow i$ (the same Row no.)

Tableau Test (cont.)

2. Repeatedly until no more change tableau T:

   For each $X \to Y \in F$

   if $\exists\ (t_1, t_2) \in T:\ t_1.X = t_2.X \wedge t_1.Y \neq t_2.Y$

   if $t_1.Y = 0$ make $t_2.Y \leftarrow 0$

   if $t_2.Y = 0$ make $t_1.Y \leftarrow 0$

   if $t_1.Y = t_1,\ t_2.Y = t_2$ make them both t_1 or t_2

3. If exits a row has become all 0 then

   ρ is *Preserve information*.

   If not, ρ is *Not Preserve information*. ■

e.g.

$R(ABCD)$, $F = \{ A \to B,\ B \to C,\ CD \to A \}$
Decomposed into $\{ R_1(AD),\ R_2(AC),\ R_3(BCD) \}$
Preserve information testing.

The initial tableau:

	A	**B**	**C**	**D**
$R_1(AD)$	0	1	1	0
$R_2(AC)$	0	2	0	2
$R_3(BCD)$	3	0	0	0

Apply $A \to B$ to equate $1, 2$. Chosing 1 as the representative symbol (Figure below).

e.g. (cont.)

	A	B	C	D
$R_1(AD)$	0	1	1	0
$R_2(AC)$	0	1	0	2
$R_3(BCD)$	3	0	0	0

$F = \{\ A \to B,\ B \to C,\ CD \to A\ \}$

Apply $B \to C$ to equate $1, 0$. Chosing 0 as the representative symbol (Figure below).

e.g. (cont.)

	A	B	C	D
$R_1(AD)$	0	1	0	0
$R_2(AC)$	0	1	0	2
$R_3(BCD)$	3	0	0	0

$F = \{\ A \to B,\ B \to C,\ CD \to A\ \}$

Apply $CD \to A$ for row 1 and row 3, to equate 0, 3. Chosing 0 as the representative symbol (Figure below).

e.g. (cont.)

	A	B	C	D
$R_1(AD)$	0	1	0	0
$R_2(AC)$	0	1	0	2
$R_3(BCD)$	0	0	0	0

The third row has all 0. Preserve information decomposition ■

e.g.

SuppInfo(Name, Addr, Item, Price),

$F = \{\ \mathit{Name}, \mathit{Item} \rightarrow \mathit{Price};\ \mathit{Name} \rightarrow \mathit{Addr}\ \}$

Decompose:

$R_1 =$ **Supp(Name, Addr)**,
$R_2 =$ **SuppDetail(Name, Item, Price)**

	Name	**Addr**	**Item**	**Price**
R_1	0	0	1	1
R_2	0	2	0	0

e.g. (cont.)

Since *Name* $\rightarrow$ *Addr* and two rows agree on *Name*,
we may equate their symbols for *Addr*, making 2 become 0.

The resulting table is:

	Name	**Addr**	**Item**	**Price**
R_1	0	0	1	1
R_2	0	0	0	0

The second row has all 0. Preserve information decomposition ■

e.g.

$R(ABCDE)$, $F = \{\ A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A\ \}$,

$R_1(AD),\ R_2(AB),\ R_3(BE),\ R_4(CDE),\ R_5(AE)$

The initial tableau:

	A	**B**	**C**	**D**	**E**
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	2	2	2
$R_3(BE)$	3	0	3	3	0
$R_4(CDE)$	4	4	0	0	0
$R_5(AE)$	0	5	5	5	0

Apply $A \to C$ for rows: 1, 2, 5 to equate 1, 2, 5. Choosing 1 as the representative symbol (Figure below).

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	2	2
$R_3(BE)$	3	0	3	3	0
$R_4(CDE)$	4	4	0	0	0
$R_5(AE)$	0	5	1	5	0

$F = \{ A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A \}$

Apply $B \to C$ for rows: 2, 3 to equate $1, 3$. Choosing 1 as the representative symbol. (Figure below)

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	2	2
$R_3(BE)$	3	0	1	3	0
$R_4(CDE)$	4	4	0	0	0
$R_5(AE)$	0	5	1	5	0

$F = \{ A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A \}$

Apply $C \to D$ for roes: 1, 2, 3, 5 to equate 0, 2, 3, 5. The result symbol must be 0. (Figure below)

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	0	2
$R_3(BE)$	3	0	1	0	0
$R_4(CDE)$	4	4	0	0	0
$R_5(AE)$	0	5	1	0	0

$F = \{ A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A \}$

Apply $DE \to C$ for rows: 3, 4, 5 to equate 1 with 0.
(Figure below)

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	0	2
$R_3(BE)$	3	0	0	0	0
$R_4(CDE)$	4	4	0	0	0
$R_5(AE)$	0	5	0	0	0

$F = \{ A \to C,\ B \to C,\ C \to D,\ DE \to C,\ CE \to A \}$

Apply $CE \to A$ for rows: 3, 4, 5 to equate 3, 4, 0 must be with 0.
(Figure below)

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	0	2
$R_3(BE)$	0	0	0	0	0
$R_4(CDE)$	0	4	0	0	0
$R_5(AE)$	0	5	0	0	0

e.g. (cont.)

	A	B	C	D	E
$R_1(AD)$	0	1	1	0	1
$R_2(AB)$	0	0	1	0	2
$R_3(BE)$	0	0	0	0	0
$R_4(CDE)$	0	4	0	0	0
$R_5(AE)$	0	5	0	0	0

The third row has all 0. Preserve information decomposition ■

Preserve Information Decomposition Theorem

Lossless Join Decomposition.

R , F;
$\rho = \{ R_1, R_2 \}$: Decomposition.

ρ has a *recovered information decomposition* if and only if:

- $(R_1 \cap R_2) \rightarrow (R_1 - R_2) \in F^+$, or
- $(R_1 \cap R_2) \rightarrow (R_2 - R_1) \in F^+$

Note: Two FDs need not be in F; it can be in F^+

Proof: The initial table is shown in below:

	$R_1 \cap R_2$	$R_1 - R_2$	$R_2 - R_1$
row for R_1	$0 \cdots 0$	$0 \cdots 0$	$1 \cdots 1$
row for R_2	$0 \cdots 0$	$2 \cdots 2$	$0 \cdots 0$

Theorem (cont.)

If $(R_1 \cap R_2) \rightarrow (R_1 - R_2) \in F^+$.
Result has the second row to be all 0.

	$R_1 \cap R_2$	$R_1 - R_2$	$R_2 - R_1$
row for R_1	$0 \cdots 0$	$0 \cdots 0$	$1 \cdots 1$
row for R_2	$0 \cdots 0$	$0 \cdots 0$	$0 \cdots 0$

If $(R_1 \cap R_2) \rightarrow (R_2 - R_1) \in F^+$.
Result has the first row to be all 0.

	$R_1 \cap R_2$	$R_1 - R_2$	$R_2 - R_1$
row for R_1	$0 \cdots 0$	$0 \cdots 0$	$0 \cdots 0$
row for R_2	$0 \cdots 0$	$2 \cdots 2$	$0 \cdots 0$

We has ρ Preserve information decomposition ■

e.g.

$R(ABC)$, $F = \{ A \to B \}$

1. R into $R_1(AB)$ and $R_2(AC)$: Preserve information.

Since $R_1 \cap R_2 = A$; $R_1 - R_2 = B$ and $A \to B$

2. R into $R_1(AB)$ and $R_2(BC)$: Not Preserve information.

Since $R_1 \cap R_2 = B$, and:

- $R_1 - R_2 = A$ and $F \not\models B \to A$
- $R_2 - R_1 = C$ and $F \not\models B \to C$

e.g. (cont.)

R into $R_1(AB)$ and $R_2(BC)$: Not Preserve information.

Considering $r(R) = \{(1,1,1),(2,1,2)\}$

A	B	C
1	1	1
2	1	2

$\pi_{AB}(r)$

A	B
1	1
2	1

$\pi_{BC}(r)$

B	C
1	1
1	2

e.g. (cont.)

$\pi_{AB}(r) \bowtie \pi_{BC}(r) = \{\ (1,1,1),\ (1,1,2),\ (2,1,1),\ (2,1,2)\ \}$

A	B	C
1	1	1
1	1	2
2	1	1
2	1	2

This natural join is proper superset of r.

Preserve Dependencise Decoposition

Preserve Dependencies

R , F.

R decomposed into $\rho = \{ R_1, R_2, \cdots, R_n \}$.

Decomposition ρ preserve F if the union of all the dependencies in $\pi_{R_i}(F)$, for $i = \overline{1..n}$ logically implies all the dependencies in F:

$$\{ \pi_{R_1}(F) \cup \pi_{R_2}(F) \cup \cdots \cup \pi_{R_n}(F) \} \models F, \text{ aka}$$

$$\bigcup_{i=1}^{n} \pi_{R_i}(F) \models F$$

Note: The converse is always true; F always implies all its projections, and therefore implies their union.

$$F \models \bigcup_{i=1}^{n} \pi_{R_i}(F)$$

e.g.

Store(Category, Warehouse, Product), with

$F = \{\ Product \rightarrow Category;\ \ Category, Warehouse \rightarrow Product\ \}$

Key: $K_1 = \{Category, Warehouse\}$,
$K_2 = \{Warehouse, Product\}$

Decopose **Store** into:

- $R_1(Category, Product)$,

  $F_1 = \{Product \rightarrow Category\},\ K = \{Product\}$

- $\mathcal{R}_2(Product, Warehouse)$,

  $F_2 = \emptyset,\ K = \{Product, Warehouse\}$

e.g. (cont.)

This is a reserve information, since: $F \models (R_1 \cap R_2) \rightarrow (R_1 - R_2)$
But this decomposition does not preserve dependencies, since

$$F_1 \cup F_2 \not\equiv F$$

R_1

(Category	Product)
1	1
1	2

R_2

(Product	Warehouse)
1	1
2	1

However, this join violates FD:
Category, *Warehouse* $\rightarrow$ *Product*

Store	(Category	Warehouse	Product)
	1	1	1
	1	1	2

Testing Preserve Dependencies

R , F , $\rho = \{R_1, \cdots, R_n\}$.

Easy to test ρ preserves F by:

1. Computing $F_i = \pi_{R_i}(F^+)$.
2. Test whether: $\bigcup_{i=1}^{n} F_i \models F$?

e.g.

$R(ABCD)$, $F = \{ A \to B,\ B \to C,\ C \to D,\ D \to A \}$. Decomposition $\rho = \{ R_1(AB),\ R_2(BC),\ R_3(CD) \}$

Project F^+ onto R_i, we get result:

$F_1 = \{ A \to B,\ B \to A \}$,

$F_2 = \{ B \to C,\ C \to B \}$,

$F_3 = \{ C \to D,\ D \to C \}$

So $\bigcup_{i=1}^{3} F_i \models F$

Preserve Information Decomposition into BCNF

Decomposition Algorithm

Input: R_0 , F_0.
Output: A decomposition of R_0 into a collection of relations, all of which are in BCNF.
Method: $R = R_0$ and $F = F_0$.
Apply recursively $< R,\ F >$.

1. If R is in BCNF, return R. Done.
2. If R violated BCNF by $X \rightarrow Y$ (X is not a super key), decomposition R into $R_1(X_F^+)$ and $R_2(X \cup (R - X_F^+))$.
3. Compute $F_1 = \pi_{R_1}(F^+)$, $F_2 = \pi_{R_2}(F^+)$.
4. Recursively decompose $< \mathcal{R}_1,\ F_1 >$ and $< R_2,\ F_2 >$ using this algorithm.
5. Return the union of the results of these decompositions.

e.g.

Instock(Product, Category, Supply, Date, Quantity), with
$F = \{$ *Product* $\rightarrow$ *Category*; *Category* $\rightarrow$ *Supply*;
Product, *Date* $\rightarrow$ *Quantity* $\}$

Key $= \{Product, Date\}$ and **Instock** is in 1NF.

1. $R =$ **Instock**
   R violated BCNF by *Product* $\rightarrow$ *Category*
   $Product^+ = \{Product, Category, Supply\}$
   $R_1(Product, Category, Supply)$,
   $F_1 = \{Product \rightarrow Category; Category \rightarrow Supply\}$
   Key $= \{Product\}$, violated BCNF by *Category* $\rightarrow$ *Supply*

   $R_2(Product \cup (R - Product^+))$, $R_2(Product, Date, Quantity)$
   $F_2 = \{Product, Date \rightarrow Quantity\}$
   Key $= \{Product, Date\}$, it is in BCNF.

e.g. (cont.)

2. $R_1(Product, Category, Supply)$
   $F_1 = \{Product \rightarrow Category;\ Category \rightarrow Supply\}$
   Key $= \{Product\}$, violated BCNF by $Category \rightarrow Supply$
   $Category^+ = \{Category, Supply\}$
   $R_{11}(Category, Supply)$
   $F_{11} = \{Category \rightarrow Supply\}$
   Key $= \{Category\}$, it is in BCNF.

   $R_{12}(Product, Category)$
   $F_{12} = \{Product \rightarrow Category\}$
   Key $= \{Product\}$, it is in BCNF.

e.g. (cont.)

Renumber relation in BCNF result:

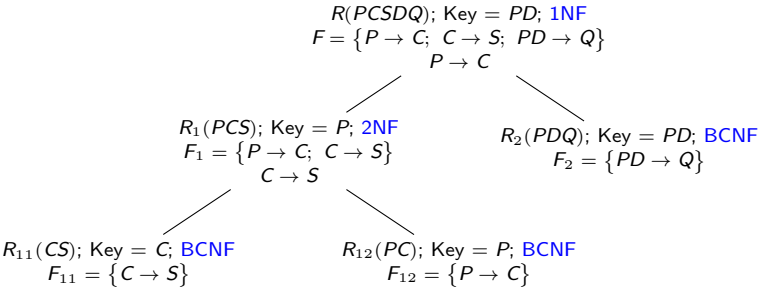
$R_1(Category, Supply)$, in BCNF
$F_1 = \{Category \rightarrow Supply\}$
Key $= \{Category\}$

$R_2(Product, Category)$, in BCNF
$F_2 = \{Product \rightarrow Category\}$
Key $= \{Product\}$

$R_3(Product, Date, Quantity)$, in BCNF
$F_3 = \{Product, Date \rightarrow Quantity\}$
Key $= \{Product, Date\}$

This result can be show by tree below.
With Product = P; Category = C; Supply = S; Date = D; Quantity = Q.

e.g. (cont.)



Notice: This algorithm decomposition into BCNF, its preserve information but not sure about preserve FDs.

Preserve Information Decomposition into 3NF

Synthesis Algorithm

It decompose into 3NF Relations With preserve information and dependency.

Input: R , F

Output: Decomposition ρ, each of which is in 3NF. It has preserve information and dependency.

Method: Perform the following steps:

1. Find a minimum cover of F, say F_c.
2. For each $X \rightarrow A \in F_c$, use XA as the scheme of one of the relations in the decomposition.
3. If none of the relation schemes from Step 2 is a superkey of R, add another relation whose scheme is a key of R.

e.g.

$R(ABCDE)$, $F = \{\ AB \to C,\ C \to B,\ A \to D\ \}$

F is also a minimum cover. So we have:

$R_1(ABC)$, $R_2(BC)$, $R_3(AD)$.

Not to use a relation whose is a proper subset of another relation, so we can drop R_2.

R has two keys: $K_1 = ABE$ and $K_2 = ACE$.
Neither of these keys is a subset of the schemes chosen so far. Thus, we must add one of them, say $R_4(ABE)$.

The final decomposition of R is thus:

$R_1(ABC)$; $F_1 = \{\ AB \to C,\ C \to B\ \}$; $3NF$
$R_2(BC)$; $F_2 = \{\ C \to B\ \}$; $BCNF$
$R_3(AD)$; $F_3 = \{\ A \to D\ \}$; $BCNF$
$R_4(ABE)$; $F_4 = \emptyset$; $BCNF$

Overview

Supply(Name, Item, Delivery)

Supply	**(Name**	**Item**	**Delivery)**
	Amazon	Laptop	Moto
	Amazon	Washer	Moto
	Amazon	Laptop	Pick-Up
	Amazon	Washer	Pick-Up
	McD	Hamb	Drone
	McD	Hamb	Scooter

∗ (n, i, d) ∈ **Supply**: supply name n supplied item i and can use delivery type t for that item.

∗ There is no FDs *Name* → *Item* or *Name* → *Delivery* on **Supply**, yet **Supply** decomposes losslessly onto:

(Name, Item) and **(Name, Delivery)**.

Overview (cont.)

SupItem

(Name	Item)
Amazon	Laptop
Amazon	Washer
McD	Hamb

SupDeli

(Name	Delivery)
Amazon	Moto
Amazon	Pick-Up
McD	Drone
McD	Scooter

SupItem ⋈ SupDeli

(Name	Item	Delivery)
Amazon	Laptop	Moto
Amazon	Laptop	Pick-Up
Amazon	Washer	Moto
Amazon	Washer	Pick-Up
McD	Hamb	Drone
McD	Hamb	Scooter

Overview (cont.)

Another instance of the relation **Supply**:

Supply	**(Name**	**Item**	**Delivery)**
	Amazon	Laptop	Moto
	Amazon	Washer	Moto
	Amazon	Washer	Pick-Up
	McD	Hamb	Drone
	McD	Hamb	Scooter

Like above instance, if we decompose this instance onto:
(Name, Item) and **(Name, Delivery)**.

Overview (cont.)

And join the two projections, *we do not get back original instance.*

SupItem (Name Item)

Amazon	Laptop
Amazon	Washer
McD	Hamb

SupDeli (Name Delivery)

Amazon	Moto
Amazon	Pick-Up
McD	Drone
McD	Scooter

SupItem ⋈ SupDeli (Name Item Delivery)

Amazon	Laptop	Moto
Amazon	Laptop	Pick-Up
Amazon	Washer	Moto
Amazon	Washer	Pick-Up
McD	Hamb	Drone
McD	Hamb	Scooter

Multivalued Dependencies

Difference of two instance above is in the first instance:
If t_1(n, i, d), t_2(n, i', d'), then t_3(n, i', d).

Definition $R(XYZ)$; X, Y be disjoint subsets of R, and

$$Z = \mathcal{R} - (XY).$$

$r(R)$ satisfies the multivalued dependency (MVD) $X \twoheadrightarrow Y$

if $\forall (t_1, t_2) \in r(R)$: $t_1.X = t_2.X$

$\exists\ t_3 \in r(R)$: $t_3.X = t_1.X = t_2.X$,
$t_3.Y = t_1.Y$, and
$t_3.Z = t_2.Z$.

The symmetry of t_1 and t_2 in this definition implies:

$\exists\ t_4 \in r(R)$: $t_4.X = t_1.X = t_2.X$,
$t_4.Y = t_2.Y$, and
$t_4.Z = t_1.Z$.

Notice: Dislike FD, this definition concern one $r(\mathcal{R})$ not $\forall r(\mathcal{R})$.

Definition (cont.)

Suppose r on $\mathcal{R}(XYZ)$ satisfies the MVD $X \twoheadrightarrow Y$

Representation of the definition below for easy remember:

r ($\mathcal{R}$:	**X**	**Y**	**Z**)
t_1:	1	1	1
t_2:	1	2	2
t_3:	1	1	2
t_4:	1	2	1

r ($\mathcal{R}$:	**X**	**Y**	**Z**)
t_1:	x	y	z
t_2:	x	y'	z'
t_3:	x	y	z'
t_4:	x	y'	z

Definition (cont.)

Note 1: The $X \twoheadrightarrow Y$ says that the relationship between X and Y is independent of the relationship between X and $(R - Y)$.

Note 2: $X \twoheadrightarrow Y$ holds in R if whenever we have:

- $(t_1, t_2) \in r(R)$: $t_1(X) = t_2(X)$, then
- We can swap two values $t_1(Y)$ *and* $t_2(Y)$ and get new (t_3, t_4) that are also $\in r(R)$.

Lemma

Relation $r(R)$ satisfies the $X \twoheadrightarrow Y$:

1. $Z = R - XY$, then $r(R)$ satisfies $X \twoheadrightarrow Z$.
2. If X, Y are not disjoin: $Y' = Y - X$, then $X \twoheadrightarrow Y'$
3. $X' \subseteq X$, then $X \twoheadrightarrow X'Y$

Proof

1. Easy to see when we change role of Y and Z
2. $Z = R - (XY) = R - (XY')$
   Since $X \twoheadrightarrow Y$
   $\forall\ (t_1, t_2) \in r(R)$: $t_1.X = t_2.X$
   $\exists t_3$: $t_3.X = t_1.X = t_2.X$,
   $t_3.Y = t_1.Y$ and $t_3.Z = t_2.Z$

   Since $Y' \subseteq Y$, so that $t_3.Y = t_1.Y \Rightarrow t_3.Y' = t_1.Y'$

   Definition implies $r(R)$ satisfies $X \twoheadrightarrow Y'$ ■

Lemma (cont.)

Relation $r(R)$ satisfies the $X \twoheadrightarrow Y$:

1. $Z = R - XY$, then $r(R)$ satisfies $X \twoheadrightarrow Z$.
2. If X, Y are not disjoin: $Y' = Y - X$, then $X \twoheadrightarrow Y'$
3. $X' \subseteq X$, then $X \twoheadrightarrow X'Y$

Proof

3. $Z = R - (XX'Y) = R - (XY)$
   Since $X \twoheadrightarrow Y$
   $\forall\ (t_1, t_2) \in r(R)$: $t_1.X = t_2.X$
   $\exists t_3$: $t_3.X = t_1.X = t_2.X$,
   $t_3.Y = t_1.Y$ and $t_3.Z = t_2.Z$

   Since $X' \subseteq X$, so that $t_3.X' = t_1.X'$, then $t_3.X'Y = t_1.X'Y$

   Definition implies $r(R)$ satisfies $X \twoheadrightarrow X'Y$ ■

e.g.

$r(R : ABCD)$ shown below satisfies the $AB \twoheadrightarrow BC$, hence it satisfies the $AB \twoheadrightarrow D$, $AB \twoheadrightarrow C$ and $AB \twoheadrightarrow ABC$

$r\ (\mathcal{R}$:	**A**	**B**	**C**	**D**)
t_1:	1	1	1	1
t_2:	1	1	2	2
t_3:	1	1	1	2
t_4:	1	1	2	1
t_5:	1	2	2	1
t_6:	2	1	1	2

Trivial MVDs

Definition

- R and $X, Y \subseteq R$
- MVD $X \twoheadrightarrow Y$ is *trivial* if any relation $r(R)$ satisfies $X \twoheadrightarrow Y$.
- So that
  - $Y \subseteq X$ or
  - $XY = R$

Theorem

$R, \quad X \subseteq R, \quad Y \subseteq R, \quad Z = R - (XY)$

$r(R)$ satisfies the $X \twoheadrightarrow Y$

$\Leftrightarrow$ $r(R)$ reserve information decomposition into $\begin{cases} r_1 : R_1(XY) \\ r_2 : R_2(XZ) \end{cases}$ **Proof**

$r(XYZ), \quad r_1 = \pi_{XY}(r), \quad r_2 = \pi_{XZ}(r)$

$r.\ X \twoheadrightarrow Y \Leftrightarrow r = r_1 \bowtie r_2$

1 "$\Rightarrow$": $r : X \twoheadrightarrow Y \Rightarrow r = r_1 \bowtie r_2$

i.e. $r : X \twoheadrightarrow Y \Rightarrow \begin{cases} r \subseteq r_1 \bowtie r_2, \\ r_1 \bowtie r_2 \subseteq r \end{cases}$

Theorem (cont.)

1.a $r \subseteq r_1 \bowtie r_2$

i.e. $\forall t \in r \Rightarrow t \in r_1 \bowtie r_2$. This is evident without to MVD:

$\forall t \in r$

$$\left.\begin{array}{l} t_1 = t.XY \Rightarrow t_1 \in \pi_{XY}(r) \\ t_2 = t.XZ \Rightarrow t_2 \in \pi_{XZ}(r) \end{array}\right\} \Rightarrow t_1 \bowtie t_2 \in \pi_{XY}(r) \bowtie \pi_{XZ}(r)$$

Since $t = t_1 \bowtie t_2 \Rightarrow t \in r_1 \bowtie r_2$.

1.b $r_1 \bowtie r_2 \subseteq r$

i.e $\forall t \in r_1 \bowtie r_2 \Rightarrow t \in r$.

$$\forall t \in r_1 \bowtie r_2 \Rightarrow \left\{\begin{array}{l} \exists t_1 \in r_1, \\ \exists t_2 \in r_2 \end{array}\right. : t = t_1 \bowtie t_2$$

i.e. $\left\{\begin{array}{l} t.XY = t_1, \\ t.XZ = t_2, \\ t_1.X = t_2.X \end{array}\right.$

Theorem (cont.)

So we have:

$$\begin{cases} t.X = t_1.X = t_2.X, & (1) \\ t.Y = t_1.Y, & (2) \\ t.Z = t_2.Z & (3) \end{cases}$$

$$t_1 \in \pi_{XY}(r) \Rightarrow \exists t'_1 \in r : t'_1.XY = t_1 \qquad (4)$$

$$t_2 \in \pi_{XZ}(r) \Rightarrow \exists t'_2 \in r : t'_2.XZ = t_2 \qquad (5)$$

$$(4,1,5) \Rightarrow t'_1.X = t_1.X = t_2.X = t'_2.X$$

$$\Rightarrow t'_1.X = t'_2.X$$

$$\left.\begin{array}{r} (4) \Rightarrow t'_1.Y = t_1.Y \\ (2) \end{array}\right\} \Rightarrow t.Y = t'_1.Y$$

Theorem (cont.)

$$\left.\begin{array}{r} (5) \Rightarrow t_2'.Z = t_2.Z \\ (3) \end{array}\right\} \Rightarrow t.Z = t_2'.Z$$

We have:

$$\begin{cases} \exists (t_1', t_2') \in r,\ t_1'.X = t_2'.X \\ \exists t \in r,\ t.X = t_1'.X = t_2'.X \\ \qquad\quad t.Y = t_1'.Y \\ \qquad\quad t.Z = t_2'.Z \end{cases}$$

This is the definition of MVD $X \twoheadrightarrow Y$ holds on $r : R(XYZ)$.
t satisfied all requirements above so that $t \in r$.

Theorem (cont.)

2 "$\Leftarrow$": $r = r_1 \bowtie r_2 \Rightarrow r : X \twoheadrightarrow Y$

$\forall (t_1, t_2) \in r : t_1.X = t_2.X$

$\exists t'_1 \in (r_1 = \pi_{XY}(r)) : t'_1 = t_1.XY \quad (1)$

$\exists t'_2 \in (r_2 = \pi_{XZ}(r)) : t'_2 = t_2.XZ \quad (2)$

Since $r = r_1 \bowtie r_2$, so:

$\exists t \in r : t = t'_1 \bowtie t'_2$

So that,

since (1): $t.XY = t'_1 = t_1.XY \quad (3)$

since (2): $t.XZ = t'_2 = t_2.XZ \quad (4)$

$(3) \Rightarrow t.X = t_1.X, t.Y = t_1.Y \quad (5)$

$(4) \Rightarrow t.X = t_2.X, t.Z = t_2.Z \quad (6)$

Theorem (cont.)

All above we have:

$$\begin{cases} \forall (t_1, t_2) \in r : t_1.X = t_2.X \\ \exists t : t.X = t_1.X = t_2.X \\ (5) \Rightarrow t.Y = t_1.Y \\ (6) \Rightarrow t.Z = t_2.Z \end{cases}$$

Follow definition r satisfied $X \twoheadrightarrow Y$ on $r : R(XYZ)$ ■

Test MVDs

$r : R(XYZ)$:

1. Theorem 3.6 gives us a method to test if a relation $r(R)$ satisfies $X \twoheadrightarrow Y$:

1. $r_1 = \pi_{XY}(r), \quad r_2 = \pi_{XZ}(r)$
2. Test if $r = r_1 \bowtie r_2$?

2. Another method: Use *only some sorting and counting.*

1. Given X-value x:
2. n_1: tuples in r_1 with X-value x
3. n_2: tuples in r_2 with X-value x
4. n: tuples in r with X-value x
5. If $n = n_1 \times n_2$ for any X-value x, then $r = r_1 \bowtie r_2$

Test MVDs (cont.)

Define the function counts the number of different W-values associated with a given X-value in r.

$$\mathcal{C}_W[X = x](r) = |\pi_W(\sigma_{X=x}(r))|$$

The condition for $X \twoheadrightarrow Y$ can be stated as:
For any X-value x in r:

$$\mathcal{C}_{\mathcal{R}}[X = x](r) \quad = \quad \mathcal{C}_{XY}[X = x](r) \quad \times \quad \mathcal{C}_{XZ}[X = x](r)$$

Since $\mathcal{C}_{WX}[X = x](r) \; = \; \mathcal{C}_W[X = x](r)$, we can simplify to:

$$\mathcal{C}_{\mathcal{R}}[X = x](r) \quad = \quad \mathcal{C}_Y[X = x](r) \quad \times \quad \mathcal{C}_Z[X = x](r)$$

e.g.

$R(ABCD)$, Test $AB \twoheadrightarrow C$

r ($\mathcal{R}$:	**A**	**B**	**C**	**D**)
t_1:	1	1	1	1
t_2:	1	1	2	2
t_3:	1	1	1	2
t_4:	1	1	2	1
t_5:	1	2	2	1
t_6:	2	1	1	2

$\mathcal{C}_{ABCD}[AB = 11](r) = 4$; $\mathcal{C}_C[AB = 11](r) = 2$; $\mathcal{C}_D[AB = 11](r) = 2$
$\mathcal{C}_{ABCD}[AB = 12](r) = 1$; $\mathcal{C}_C[AB = 12](r) = 1$; $\mathcal{C}_D[AB = 12](r) = 1$
$\mathcal{C}_{ABCD}[AB = 21](r) = 1$; $\mathcal{C}_C[AB = 21](r) = 1$; $\mathcal{C}_D[AB = 21](r) = 1$

Inference Axioms MVDs Alone

$r(\mathcal{R})$; W, X, Y, Z be subsets of R.

M1. Reflexivity

$$Y \subseteq X \models X \twoheadrightarrow Y$$

M2. Augmentation

$$\left.\begin{array}{l} X \twoheadrightarrow Y \\ W \subseteq Z \end{array}\right\} \models XZ \twoheadrightarrow YW$$

M3. Transitivity

$$\left.\begin{array}{l} X \twoheadrightarrow Y \\ Y \twoheadrightarrow Z \end{array}\right\} \models X \twoheadrightarrow Z - Y$$

M4. Additivity

$$\left.\begin{array}{l} X \twoheadrightarrow Y \\ X \twoheadrightarrow Z \end{array}\right\} \models X \twoheadrightarrow YZ$$

Inference Axioms MVDs Alone (cont.)

M5. Decomposition

$$\left.\begin{array}{l} X \twoheadrightarrow Y \\ X \twoheadrightarrow Z \end{array}\right\} \models \left\{\begin{array}{l} X \twoheadrightarrow Y \cap Z \\ X \twoheadrightarrow Y - Z \\ X \twoheadrightarrow Z - Y \end{array}\right.$$

M6. Pseudo Transitivity

$$\left.\begin{array}{l} X \twoheadrightarrow Y \\ WY \twoheadrightarrow Z \end{array}\right\} \models XW \twoheadrightarrow Z - (YW)$$

M7. Complementation

$$\left.\begin{array}{r} X \twoheadrightarrow Y \\ Z = \mathcal{R} - (XY) \end{array}\right\} \models X \twoheadrightarrow Z$$

Inference Axioms FDs and MVDs

$r(\mathcal{R})$; W, X, Y, Z be subsets of R.

C1. Replication

$$X \to Y \models X \twoheadrightarrow Y$$

C2. Coalescence

$$\left.\begin{array}{r} X \twoheadrightarrow Y \\ \exists W,\ W \cap Y = \emptyset \\ W \to Z \\ Z \subseteq Y \end{array}\right\} \models X \to Z$$

e.g.

$R(ABCDE)$, $D = \{\ A \to BC,\ DE \twoheadrightarrow C\ \}$

$D \models A \twoheadrightarrow BDE$?

$A \to BC \models A \twoheadrightarrow BC$ (C1)

$A \twoheadrightarrow BC \models A \twoheadrightarrow DE$ (M7)

$$\left.\begin{array}{l} A \twoheadrightarrow DE \\ DE \twoheadrightarrow C \end{array}\right\} \models A \twoheadrightarrow C \text{ (M3)}$$

$A \twoheadrightarrow C \models A \twoheadrightarrow BDE$ (M7) ■

Finding X_F^+ by Chase

R , F , $X \subseteq R$;
Compute X_F^+ all attributes that functionally depend on X.

1. Start with a tableau having two rows that agree only on X.
2. Chase the tableau using the FDs of F.
3. The columns in final tableau agrees two rows is X_F^+.

e.g.
$R(ABCDEH)$, $F = \{\ AB \to C,\ BC \to AD,\ D \to E,\ CH \to B\ \}$

$F \models AB \to D$?
Start with the tableau:

A	B	C	D	E	H
0	0	1	1	1	1
0	0	2	2	2	2

Finding X_F^+ by Chase (cont.)

Apply $AB \to C$ to infer $1 = 2$; say both become 1. The resulting tableau is:

A	B	C	D	E	H
0	0	1	1	1	1
0	0	1	2	2	2

Next, apply $BC \to AD$ to infer that $1 = 2$, and apply $D \to E$ to infer $1 = 2$. At this point, the tableau is:

A	B	C	D	E	H
0	0	1	1	1	1
0	0	1	1	1	2

Since the two tuples agree in the D column implies $F \models AB \to D$.
Other hand: $AB_F^+ = (ABCDE)$.
In this tableau, only one AB_F^+ not other attributes.

Tableau Chase Test for MVDs

$X \twoheadrightarrow Y$ holds in $r : \mathcal{R}(XYZ)$ if only if:

$$r(XYZ) = r_1(XY) \bowtie r_2(XZ)$$

So we can use Tableau to check $r_1(XY)$ and $r_2(XZ)$ reserve information with $r(XYZ)$ whether implies $X \twoheadrightarrow Y$.

e.g.
$R(ABCD)$, $\mathfrak{D} = \{ A \to B,\ B \twoheadrightarrow C \}$

$\mathfrak{D} \models A \twoheadrightarrow C$ holds in $r(R)$?

Start with tableau:

R	**A**	**B**	**C**	**D**
$r_1(AC)$	0	1	0	1
$r_2(ABD)$	0	0	2	0

Tableau Chase Test for MVDs (cont.)

Apply $A \rightarrow B$ to infer that $1 = 0$.
The tableau becomes:

R	**A**	**B**	**C**	**D**
$r_1(AC)$	0	0	0	1
$r_2(ABD)$	0	0	2	0

Apply the $B \twoheadrightarrow C$, since two rows agree in the B column. We swap C columns to get two more rows which we add to the tableau, which becomes:

R	**A**	**B**	**C**	**D**
$r_1(AC)$	0	0	0	1
$r_2(ABD)$	0	0	2	0
	0	0	2	1
	0	0	0	0

A row with all 0, so that $F \models A \twoheadrightarrow C$ ■

e.g.

$R(ABCGHI)$, $\mathfrak{D} = \{\ A \twoheadrightarrow B,\ B \twoheadrightarrow HI,\ CG \twoheadrightarrow H\ \}$

$\mathfrak{D} \models AB \twoheadrightarrow BH$

Using Tableau Chase Test:

R	**A**	**B**	**C**	**G**	**H**	**I**	
$r_1(AGBH)$	0	0	1	0	0	1	t_1
$r_2(AGCI)$	0	2	0	0	2	0	t_2
$A \twoheadrightarrow B$	0	0	0	0	2	0	t_3
using (t_1, t_2)	0	2	0	0	2	0	t_4
$B \twoheadrightarrow HI$	0	0	1	0	2	0	t_5
using (t_1, t_3)	0	0	0	0	0	1	t_6
$CG \twoheadrightarrow H$	0	0	0	0	0	0	t_7
using (t_3, t_6)	0	0	0	0	2	1	t_8

There is a row with all 0, so that $\mathfrak{D} \models AB \twoheadrightarrow BH$ ■

Projecting MVDs

$R(ABCDE)$, $\mathfrak{D} = \{ A \twoheadrightarrow CD \}$;
Decomposition $S(ABC)$. Finding MVDs imply in $S(ABC)$?

MVDs maybe have left side are A or B or C, since orther trivial MVDs.
Furthermore, we claim $A \twoheadrightarrow C$ holds in $\mathcal{S}$, as does $A \twoheadrightarrow B$ (by the complementation rule).

Start with the tableau:

R	**A**	**B**	**C**	**D**	**E**
$r_1(AC)$	0	1	0	1	1
$r_2(ABDE)$	0	0	2	0	0

Projecting MVDs (cont.)

Use $A \twoheadrightarrow CD$ to swap the C and D components of these two rows to get two new rows:

R	**A**	**B**	**C**	**D**	**E**
$r_1(AC)$	0	1	0	1	1
$r_2(ABDE)$	0	0	2	0	0
	0	1	2	0	1
	0	0	0	1	0

Notice: The last row has 0 in all the attributes of S, that is, A, B, and C. That is enough to conclude that $A \twoheadrightarrow C$ holds in S ■

Minimum Dependency Basis

Algorithm 1 David Maier (The Theory of Relational Databases)

Input: R, $\mathfrak{D} = \{\ FDs,\ MVDs\ \}$, $X \subseteq R$

Output: Minimum dependency basis for X base on $\mathfrak{D}$

Denote: $X^{++}_{\mathfrak{D}}$

Method:

1. – Change all FDs in $\mathfrak{D}$ to MVDs
   – $X^{++}_{\mathfrak{D}} \leftarrow \{A_1,\ A_2, \ldots\ A_k\}$, $A_i \in X$
   – $\forall X' \twoheadrightarrow Y' \in \mathfrak{D}\ :\ X' \subseteq X$

   $$X^{++}_{\mathfrak{D}} \leftarrow X^{++}_{\mathfrak{D}}\ +\ \{Y'\}\ +\ \{R - X'Y'\}$$

2. $\forall \{Y_1, Y_2\} \in X^{++}_{\mathfrak{D}} : Y_1 \cap Y_2 \neq \emptyset :$

   $$X^{++}_{\mathfrak{D}} \leftarrow X^{++}_{\mathfrak{D}}\ -\ \{Y_1, Y_2\}\ +\ \{Y_1 \cap Y_2\}\ +\ \{Y_1 - Y_2\}\ +\ \{Y_2 - Y_1\}$$

Minimum Dependency Basis (cont.)

3. $\forall W \twoheadrightarrow Z \in \mathfrak{D} \ : \ W \subseteq Y = Y_1 Y_2 \ \ldots \ Y_k \ , \ Y_i \in X_{\mathfrak{D}}^{++}$
   - $\mathfrak{D} \models Y \twoheadrightarrow Z$ (augmentation)
   - $\mathfrak{D} \models X \twoheadrightarrow \{Z - Y\}$ (transitivity)
   - If $\{Z - Y\}$ is NOT the union of some sets in $X_{\mathfrak{D}}^{++}$:

     $$X_{\mathfrak{D}}^{++} \leftarrow X_{\mathfrak{D}}^{++} + \{Z - Y\}$$
4. If no MVD in $\mathfrak{D}$ can be used to change $X_{\mathfrak{D}}^{++}$, stop.
   If not, turn to step 2

Like X_F^+ for FDs, with $X_{\mathfrak{D}}^{++}$ we can implies all MVDs with which left side is X and right side is subset of $X_{\mathfrak{D}}^{++}$

e.g.

$R(ABCDE)$, $\mathfrak{D} = \{A \to BC,\ DE \twoheadrightarrow C\}$
Find $A^{++}_{\mathfrak{D}}$.

1. – Change to MVDs: $\mathfrak{D} = \{A \twoheadrightarrow BC,\ DE \twoheadrightarrow C\}$
   – Initial $A^{++}_{\mathfrak{D}} = \{A\}$
   – $A \twoheadrightarrow BC$:
   $A^{++}_{\mathfrak{D}} = \{A,\ BC,\ DE\}$
2. None
3. $DE \twoheadrightarrow C$ with $Y = DE$:
   $A^{++}_{\mathfrak{D}} = \{A,\ BC,\ DE,\ C\}$
2. $Y_1 = BC,\ Y_2 = C$:
   $A^{++}_{\mathfrak{D}} = \{A,\ B,\ C,\ DE\}$
3. None 2. None 3. None

Result: $A^{++}_{\mathfrak{D}} = \{A,\ B,\ C,\ DE\}$

e.g.

$R(ABCDE)$, $\mathfrak{D} = \{A \to BC,\ DE \twoheadrightarrow C\}$

Find $AD^{++}_{\mathfrak{D}}$.

1. – Change to MVDs: $\mathfrak{D} = \{A \twoheadrightarrow BC,\ DE \twoheadrightarrow C\}$
   – Initial $AD^{++}_{\mathfrak{D}} = \{A,\ D\}$
   – $A \twoheadrightarrow BC$:
   $AD^{++}_{\mathfrak{D}} = \{A,\ D,\ BC,\ DE\}$
2. $Y_1 = D,\ Y_2 = DE$:
   $AD^{++}_{\mathfrak{D}} = \{A,\ D,\ E,\ BC\}$
3. $DE \twoheadrightarrow C$ with $Y = DE$:
   $AD^{++}_{\mathfrak{D}} = \{A,\ D,\ E, BC,\ C\}$
2. $Y_1 = BC,\ Y_2 = C$:
   $AD^{++}_{\mathfrak{D}} = \{A,\ D,\ E,\ B,\ C\}$

e.g. (cont.)

3. None
2. None
3. None

Result: $AD^{++}_{\mathfrak{D}} = \{A,\ B,\ C,\ D,\ E\}$

So that: $\mathfrak{D} \models AD \twoheadrightarrow BE$

We can test this MVD by use the *Axioms or Tableau and Chase.*

Another Algorithm

Minimum Dependency Basis, Beeri [1980]

Input: R, $\mathfrak{D} = \{ \ FDs, \ MVDs \ \}$, $X \subseteq R$

Output: Minimum dependency basis for X base on $\mathfrak{D}$

Denote: $X_{\mathfrak{D}}^{++}$

Method:

1. Change all FDs in $\mathfrak{D}$ to MVDs

   $X_{\mathfrak{D}}^{++} \leftarrow \{ \ R - X \ \}$
2. While (No more change $X_{\mathfrak{D}}^{++}$) Do

   For each $V \twoheadrightarrow W \in \mathfrak{D}$

   If ($\exists Y \in X_{\mathfrak{D}}^{++}$: $Y \cap W \neq \emptyset$, $Y \cap V = \emptyset$) then

   $$X_{\mathfrak{D}}^{++} \leftarrow X_{\mathfrak{D}}^{++} \ - \ \{Y\} \ + \ \{Y \cap W\} \ + \ \{Y - W\}$$
3. $X_{\mathfrak{D}}^{++}$ is dependency basis of X

Fourth Normal Form

Definition

R , $\mathfrak{D} = \{FDs,\ MVDs\}$

R is in fourth normal form (4NF) if for every $X \twoheadrightarrow Y$:

- $X \twoheadrightarrow Y$ is a *trivial* MVD or
- X is a *superkey* for R

e.g.
$R(ABCDE)$, $\mathfrak{D} = \{A \to BC,\ C \twoheadrightarrow DE\}$.

R is not in 4NF because of the $C \twoheadrightarrow DE$.

R consisting of the two $R_1(ABC)$ and $R_2(CDE)$ is in 4NF with respect to $\mathfrak{D}$,

even though $A \twoheadrightarrow B$ is implied by $\mathfrak{D}$ and applies to R_1.

$A \twoheadrightarrow B$ is not trivial, but A is a key for R_1.

Lemma

R , $\mathfrak{D} = \{FDs,\ MVD\}$. If R is in 4NF, then R is in BCNF.

Proof (use $\mathscr{C}_1 \Rightarrow \mathscr{C}_2 \Leftrightarrow \neg\mathscr{C}_2 \Rightarrow \neg\mathscr{C}_1$)
Suppose R is not in BCNF. Then we must have subsets K, Y, and A of R such that K is a key for $\mathcal{R}$ and:

1. $K \rightarrow Y$,
2. $Y \nrightarrow K$,
3. $Y \rightarrow A$,
4. $A \notin (KY)$

We try to prove $Y \twoheadrightarrow A$ is not trivial and Y is not a superkey.

– $Y \rightarrow A \models Y \twoheadrightarrow A$
  Since $A \notin Y$ and $YA \neq \mathcal{R}$, so $Y \twoheadrightarrow A$ is not trivial.
– $Y \nrightarrow K$, so Y is not a key $\Rightarrow$ Y not a superkey

Therefore, R is not in 4NF ■

Decomposition into 4NF

Quite analogous to the BCNF decomposition algorithm.

Algorithm Decomposition into 4NF.

Input: R_0 with $\mathfrak{D}_0 = \{FDs,\ MVDs\}$
Output: $\rho = \{R_1, R_2, \ldots, R_k\}$ in 4NF, conserve information
Method: Do the following steps, with $R = R_0$ and $\mathfrak{D} = \mathfrak{D}_0$

1. Find a 4NF violation in R; $X \twoheadrightarrow Y$ in R, X is not a superkey.
   If none, return True; R by itself is a suitable decomposition.
2. If have a 4NF violation, break R into two:
   - $R_1(XY)$
   - $R_2(X(R - XY))$

Decomposition into 4NF (cont.)

3. Find the FD's and MVD's: $\mathfrak{D}_1 = \pi_{R_1}(\mathfrak{D})$ and $\mathfrak{D}_2 = \pi_{R_2}(\mathfrak{D})$.
4. Recursively decompose $\{R_1, \mathfrak{D}_1\}$ and $\{R_2, \mathfrak{D}_2\}$

e.g.
$R(ABCDEI)$, $\mathfrak{D} = \{A \twoheadrightarrow BCD,\ B \to AC,\ C \to D\}$.

- Key $= (BEI)$. R is not in 4NF because of $A \twoheadrightarrow BCD$ is a nontrival MVD and A is not a key for R
- Decompose R into:
  $R_1(ABCD)$, $\mathfrak{D}_1 = \{B \to AC,\ C \to D\}$, key $= B$, 2NF
  $R_2(AEI)$, $\mathfrak{D}_2 = \emptyset$, 4NF
- Use $C \twoheadrightarrow D$ for $R_1(ABCD)$:
  $R_{11}(CD)$, $\mathfrak{D}_{11} = \{C \to D\}$, key $= C$, 4NF
  $R_{12}(ABC)$, $\mathfrak{D}_{12} = \{B \to AC\}$, key $= B$, 4NF

Result: $\rho = \{\mathcal{R}_{11}, \mathcal{R}_{12}, \mathcal{R}_2\}$ is thus in 4NF rescpect to $\mathfrak{D}$.

Overview

For example, we have $r(ABC)$ decomposes conserve information onto AB, AC and BC.

r	(A	B	C)
	1	1	1
	1	2	2
	3	3	3
	4	3	4
	5	5	5
	6	6	5

$r = \pi_{AB}(r) \bowtie \pi_{AC}(r) \bowtie \pi_{BC}(r)$

e.g. (cont.)

$\pi_{AB}(r) =$

(A	B)
1	1
1	2
3	3
4	3
5	5
6	6

$\pi_{AC}(r) =$

(A	C)
1	1
1	2
3	3
4	4
5	5
6	5

$\pi_{BC}(r) =$

(B	C)
1	1
2	2
3	3
3	4
5	5
6	5

However, r satisfies no nontrivial MVDs, so it has no conserve information decomposition onto any pair of R_1 and R_2 such that $R_1 \neq ABC$ and $R_1 \neq ABC$.

Join Dependency

$R = \{R_1, R_2, ..., R_p\}$ be a set of relation schemes over $U = R_1 R_2 ... R_p$. A relation $r(U)$ satisfies

Join Dependency: (JD) $*[R_1, R_2, ..., R_p]$

if r decomposes conserve information onto $R_1, R_2, \cdots, R_p$. That is,

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \cdots \bowtie \pi_{R_p}(r)$$

We also write $*[R_1, R_2, \cdots, R_p]$ as $*[R]$

e.g.
In example 11.1, r satisfies the JD $*[AB,\ AC,\ BC]$

Join Dependency (cont.)

We know:

1. $r(R)$ satisfies $X \twoheadrightarrow Y \Leftrightarrow r$ decomposes conserve information onto XY and XZ, where $Z = R - (XY)$.
2. This condition is just JD $*[XY, XZ]$.
3. JD $*[R_1, R_2]$ is the same as the $R_1 \cap R_2 \twoheadrightarrow R_1$.
4. We can also define JDs in a manner similar to the definition of MVDs.
5. Let r satisfy $*[R_1, R_2, \cdots, R_p]$. If r contains tuples $t_1, t_2, \cdots, t_p$ such that
$$t_i(R_i \cap R_j) = t_j(R_i \cap R_j)$$
for all i, j,
r must contain t such that $t(R_i) = t_i(R_i)$, $1 \leqslant i \leqslant p$.

e.g.

Suppose relation $r(ABCDE)$ satisfies the JD $*[ABC,\ BD,\ CDE]$ and contains the three tuples shown below.

r	**(A**	**B**	**C**	**D**	**E)**
t_1	1	1	1	1	1
t_2	2	1	2	2	3
t_3	3	2	1	2	2

r must also contain tuple $t = (1,\ 1,\ 1,\ 2,\ 2)$

JD $*[R_1, R_2, \cdots, R_p]$ over R is *trivial* if $R = R_1$ for some i.

JD $*[R_1, R_2, \cdots, R_p]$ *applies* to R if $R = R_1 R_2 \cdots R_p$.

Project Join Normal Form

R, $F = \{FDs,\ JDs\}$ over R.
R is in *project join normal form* (PJNF) with respect to F
if for every JD $*[R_1, R_2, \cdots, R_p]$ implied by F, that implies to R, and:

- JD is trivial or,
- Every R_i is a superkey for R.

e.g. Let $R(ABCDEI)$,
$F = \{ *[ABCD, CDE, BDI], *[AB, BCD, AD], A \to BCDE, BC \to AI\}$.

Key $= (A), (BC)$
R is not PJNF because of JD $*[ABCD, CDE, BDI]$.
If decompose $\rho = \{R_1(ABCD), R_2(CDE), R_3(BDI)\}$

Project Join Normal Form (cont.)

$\rho = \{R_1(ABCD), R_2(CDE), R_3(BDI)\}$

$R_1(ABCD)$, $F_1 = \{ * [AB, BCD, AD],\ A \to BCD,\ BC \to A\}$

Key $= (A)$, BC. So R_1 is in PJNF.

$R_2(CDE)$, $\mathcal{F}_2 = \emptyset$, PJNF.

$R_3(BDI)$, $F_3 = \emptyset$, PJNF.

Embedded Functional Dependencies

Given R, $X \rightarrow Y$ holds on R.
If any $S \supseteq R$. Whether $X \rightarrow Y$ holds on S ?

This is evident for FDs because the definition of FDs did not concert to other XY set.

r (R: **X** **Y**)

1	2
1	2
3	4
3	4

r (S: **X** **Y** **Z** **W**)

1	2	☺	☹
1	2	🚹	🚹
3	4	👩	🤵
3	4	☯	☹

Embedded Multivalued Dependencies

Consider the relation $r(ABCD)$. The projection $\pi_{ABC}(r)$ satisfies $A \twoheadrightarrow B$, but r itself does not.

r

(A	B	C)
1	1	1
1	2	1
1	1	2
1	2	2
2	2	2

r

(A	B	C	D)
1	1	1	1
1	2	1	1
1	1	2	2
1	2	2	1
2	2	2	2

Definition: Relation $r(R)$ satisfies the embedded multivalued dependency $X \twoheadrightarrow Y|Z$ if the $X \twoheadrightarrow Y$ is satisfied by the $\pi_{X \cup Y \cup Z}(r)$.

Read: 'X multivalued determines Y in the context of Z'.

Embedded Join Dependencies

Multivalued dependencies is spectial of Join dependencies.
$r(R)$, $X \twoheadrightarrow Y$.
Another description is JD $*[XY, XZ]$ with $Z = R - XY$.

Definition

- Relation $r(R)$ satisfies the embedded join dependency (EJD) $*[R_1, R_2, ..., R_p]$ if: $\pi_S(r)$ satisfies $*[R_1, R_2, ..., R_p]$ as a regular JD, where $S = R_1 R_2 ... R_p$.
- Allow $R = S$. That is, every JD is an EJD.
- Also write the embedded multivalued dependency (EMVD) $*[XY, XZ]$ as $X \twoheadrightarrow Y|Z$.

Overview

Objectives of physical design:

- How to translate entities into physical tables
- What attributes to use for columns of the physical tables
- Which columns of the tables to define as keys
- What indexes to define on the tables
- What views to define on the tables
- How to denormalize the tables
- How to resolve many-to-many relationships
- What designs can take advantage of hash access

Conceptual to Physical

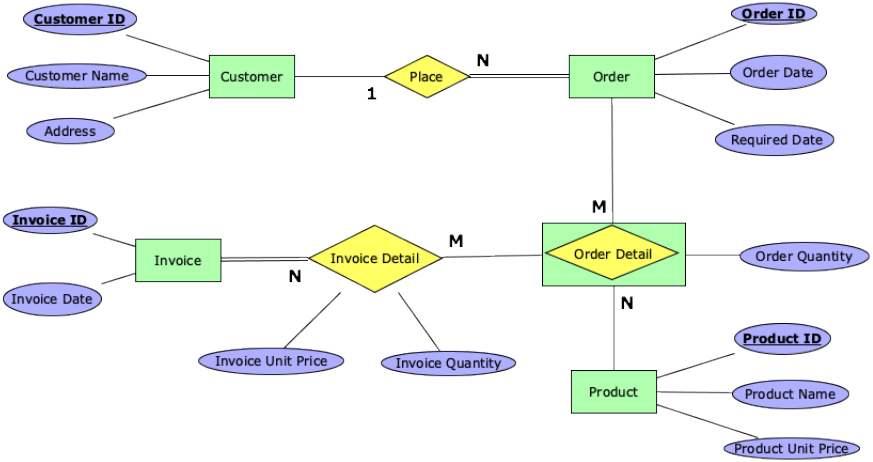
The following steps convert from conceptual model to physical model:

1. ERD to Relational model
2. Conceptual to Functional dependencies
3. Normorlization

Problem

- A construction materials store wants to manage Product including names and unit prices.
- List of ordering customers or potential customers with customer name and address information.
- Every time an order is placed by a customer, the order needs to store the order date, pick-up date, and customer. Order along with a list of products ordered and quantity ordered.
- When delivery is due, issue a delivery invoice knowing that each order can be delivered in multiple deliveries with different delivery quantities and each Order there is an invoice for billing.
- The unit price at delivery may be different from the reference price in stock at the time of ordering.

ERD


Customer ID
Customer Name
Address
Customer
1
Place
N
Order
Order ID
Order Date
Required Date
M
Invoice ID
Invoice Date
Invoice
N
Invoice Detail
M
Order Detail
Order Quantity
N
Invoice Unit Price
Invoice Quantity
Product ID
Product Name
Product
Product Unit Price

Relational Model

- Customer(Customer ID, Customer Name, Address)
- Order(Order ID, Order Date, Required Date, Customer ID)
- Product(Product ID, Product Name, Product Unit Price)
- Order Detail(Order ID, Product ID, Order Quantity)
- Invoice(Invoice ID, Invoice Date)
- Invoice Detail(Invoice ID, Order ID, Product ID, Invoice Unit Price, Invoice Quantity)

Functional Dependencies

- *Custemer ID* $\rightarrow$ *Customer Name*, *Address*
- *Order ID* $\rightarrow$ *Order Date*, *Required Date*
- *Order ID* $\rightarrow$ *Customer ID*
- *Product ID* $\rightarrow$ *Product Name*, *Product Unit Price*
- *Order ID*, *Product ID* $\rightarrow$ *Order Quantity*
- *Invoice ID* $\rightarrow$ *Invoice Date*
- *Invoice ID* $\rightarrow$ *Order ID*
- *Invoice ID*, *Order ID*, *Product ID* $\rightarrow$ *Invoice Unit Price*, *Invoice Quantity*
- *Invoice ID*, *Product ID* $\rightarrow$ *Invoice Unit Price*, *Invoice Quantity*

Normal Form

- Customer(Customer ID, Customer Name, Address) **BCNF**
- Order(Order ID, Order Date, Required Date, Customer ID) **BCNF**
- Product(Product ID, Product Name, Product Unit Price) **BCNF**
- Order Detail(Order ID, Product ID, Order Quantity) **BCNF**
- Invoice(Invoice ID, Invoice Date) **BCNF**
- Invoice Detail(Invoice ID, Order ID, Product ID, Invoice Unit Price, Invoice Quantity) **1NF**

Normalization

Normalization for:

- Invoice Detail(Invoice ID, Order ID, Product ID, Invoice Unit Price, Invoice Quantity) **1NF**

Result:

- R_1(Invoice ID, Order ID) **BCNF**
- R_2(Invoice ID, Product ID, Invoice Unit Price, Invoice Quantity) **BCNF**

Result of Normalization

- Customer(Customer ID, Customer Name, Address) **BCNF**
- Order(Order ID, Order Date, Required Date, Customer ID) **BCNF**
- Product(Product ID, Product Name, Product Unit Price) **BCNF**
- Order Detail(Order ID, Product ID, Order Quantity) **BCNF**
- Invoice(Invoice ID, Invoice Date, Order ID) **BCNF**
- Invoice Detail(Invoice ID, Product ID, Invoice Unit Price, Invoice Quantity) **BCNF**

Customize Name for SQL

- Customer(CustomerID, CustomerName, Address)
- Order(OrderID, OrderDate, RequiredDate, CustomeID)
- Product(ProductID, ProductName, ProductUnit Price)
- Order Detail(Order ID, Product ID, Order Quantity)
- Invoice(InvoiceID, InvoiceDate, OrderID)
- InvoiceDetail(InvoiceID, ProductID, InvoiceUnitPrice, InvoiceQuantity)

Foreign Keys

- Customer(CustomerID, CustomerName, Address)
- Order(OrderID, OrderDate, RequiredDate, *CustomeID*)
- Product(ProductID, ProductName, ProductUnitPrice)
- OrderDetail(*OrderID*, *ProductID*, OrderQuantity)
- Invoice(InvoiceID, InvoiceDate, *OrderID*)
- InvoiceDetail(*InvoiceID*, *ProductID*, InvoiceUnitPrice, InvoiceQuantity)

SQL Script

```
Create table Customer
(
  CustomerID char(3) primary key,
  CustomerName varchar(30),
  Address varchar(30)
);
Create table Order
(
  OrderID char(3) primary key,
  OrderDate date,
  RequiredDate date,
  CustomerID char(3) not null,
  foreign key (CustomerID) references Customer(CustomerID)
);
```

SQL Script

```
Create table Product
(
  ProductID char(3) primary key,
  ProductName varchar(30),
  ProductUnitPrice float
);
Create table OrderDetail
(
  OrderID char(3),   ProductID char(3),
  OrderQuantity float,
  primary key (OrderID, ProductID),
  foreign key (OrderID) references Order(OrderID),
  foreign key (ProductID) references Product(ProductID),
);
```

SQL Script

```
Create table Invoice
(
  InvoiceID char(3) primary key,
  InvoiceDate date,
  OrderID char(3) not null,
  foreign key (OrderID) references Order(OrderID),
);
Create table InvoiceDetail
(
  InvoiceID char(3),  ProductID char(3),  InvoiceUnitPrice float,  InvoiceQuantity float,
  primary key (InvoiceID, ProductID),
  foreign key (InvoiceID) references Invoice(InvoiceID),
  foreign key (ProductID) references Product(ProductID),
);
```

Denormalization

- The rules of normalization do not consider performance.
- In some cases, you need to consider denormalization to improve performance.

e.g.

- Customer(<u>CustomerID</u>, CustomerName, Address)
- Order(<u>OrderID</u>, OrderDate, RequiredDate, $\overline{CustomeID}$)

Change:

- Customer(<u>CustomerID</u>, CustomerName, **Address**)
- Order(<u>OrderID</u>, OrderDate, RequiredDate, $\overline{CustomeID}$, **Address**)

Pros: Don't waste time connecting, quickly retrieve customer addresses

Cons: Order: 2NF $\Rightarrow$ Check: $CustomerID \rightarrow Address$;
data redundancy and inconsistency

Customized data views

- No single table contains all the data that we need.
- Data might be scattered among several tables.
- One table might contain more data than users want to see or more than you want to authorize them to see. For those situations, you can create views.

Using views for a variety of reasons:

- To limit access to certain kinds of data
- To combine data from multiple tables

e.g.

```
CREATE VIEW OrderIncome AS
  SELECT OrderID, CustomerName, SUM(OrderQuantity * ProductUnitPrice) Total
  FROM OrderDetails INNER JOIN Orders USING (OrderID)
        INNER JOIN Customer USING (CustomerID)
  GROUP BY OrderID ;
```

Best Practices for Indexes: The Pros

- **Faster Queries:** An index is like a book's table of contents. Instead of reading the entire book (your data table), it points you directly to the information you need, making searches much faster.
- **Smoother Performance:** With an index, the computer has to do less work to find data, which reduces the load on your system.
- **Ensures Data Accuracy:** A unique index prevents you from adding duplicate records, ensuring your data remains clean and correct.
- **Efficient Data Connections:** Indexes make it quicker to connect related data tables, especially when you are using foreign key constraints.

Index Best Practices: The Cons

- **Extra Storage:** Every index you create needs its own space on the hard drive, and its size grows along with your data.
- **Slower Writes:** When you add, delete, or update a record, the database has to take extra time to update all of the related indexes.
- **Decreased Efficiency:** Maintaining too many indexes can make the database's fundamental operations, such as backups and restores, more complex and slow.

Basic Concepts

create index:

```
CREATE [ UNIQUE ] INDEX indexname ON table
    [ USING indextype ] ( column [ opclass ] [, ...] )
```

Common PostgreSQL Index Types

1. **B-Tree indexes (default index type in PostgreSQL)**
   - Default index type in PostgreSQL.
   - Supports $<$, $<=$, $=$, $>=$, $>$, **BETWEEN, IN, IS NULL, IS NOT NULL**.
   - Organizes entries in ascending order.

   CREATE INDEX index_product_id ON products (product_id);

2. **Hash Index**
   - Ideal for equality checks, especially for integers.
   - Doesn't support range queries or sorting.

   CREATE INDEX index_product_id ON products USING HASH (product_id);

Common PostgreSQL Index Types

3. **Partial indexes**
   - Built over a subset of a table, defined by a conditional expression.
   - Useful for filtering out frequently unqueried rows.

   CREATE INDEX index_product_id ON products(product_id) where product_available = 'true';

4. **Covering indexes**
   - Allows index-only scans when the select list matches index columns.
   - Additional columns are specified with the INCLUDE keyword.

   CREATE INDEX index_product_id_name_status ON products (product_id, product_name) include (status);

Common PostgreSQL Index Types

5. **Block Range index (BRIN)**
   - Designed for large, sorted tables like time-series data.
   - Space-efficient, storing min and max values within logical blocks.

```
CREATE INDEX brin_example_index ON logs USING BRIN(log_date);
```

6. **GiST indexes**
   - GiST (Generalized Search Tree) supports indexing of complex data types such as geometric shapes and full-text documents.
   - Allows for custom operators for querying, supporting advanced search functionalities like natural language processing.
   - Uses "lossy" indexing strategies to summarize complex items, potentially leading to false positives that require detailed row checks.

```
CREATE INDEX idx_gist ON my_table USING gist(my_column);
```

Common PostgreSQL Index Types

7. **SP-GiST indexes**
   - SP-GiST (Space Partitioned Generalized Search Tree) is ideal for data with natural partitioning, such as spatial and hierarchical data.
   - Enables different branching factors at different nodes, allowing tailored tree structures that reduce search space.
   - Particularly efficient for applications requiring complex spatial queries or managing large data hierarchies.

```
CREATE INDEX idx_spgist ON my_table USING spgist(my_column);
```

Multicolumn Indexes

An index can be defined on more than one column of a table. For example, if you have a table of this form:

```
CREATE TABLE test
(
    major int,
    minor int,
    name varchar
);
```

Frequently issue queries like:

SELECT name FROM test WHERE major = constant AND minor = constant;

Define an index on the columns major and minor together

CREATE INDEX test_mm_idx ON test (major, minor);

Indexes and ORDER BY

By default, B-tree indexes store their entries in ascending order with nulls last.

Order by default:

ORDER BY x ASC NULLS LAST

ORDER BY x DESC NULLS FIRST

We can adjust the ordering of a B-tree index by including the options ASC, DESC, NULLS FIRST, and/or NULLS LAST when creating the index:

```
CREATE INDEX test_info_nulls_low ON test (info NULLS FIRST);
CREATE INDEX test_desc_index ON test (id DESC NULLS LAST);
```

Combining Multiple Indexes

Index On (A, B)

We can use on postgreSQL:

WHERE A = 2 And B = 3

WHERE A = 2 Or B = 3

Indexes on Expressions

Unique Indexes

CREATE UNIQUE INDEX name ON table (column [, ...]) [NULLS [NOT] DISTINCT];

e.g.

```
SELECT * FROM test WHERE lower(col1) = 'value';
```

```
CREATE INDEX test_lower_col1_idx ON test (lower(col1));
```

e.g.

```
SELECT * FROM people
WHERE (first_name || ' ' || last_name) = 'John Smith';
```

```
CREATE INDEX people_names ON people ((first_name || ' ' || last_name));
```

Partial Indexes

A partial index is an index built over a subset of a table; the subset is defined by a conditional expression (called the predicate of the partial index)

```
CREATE INDEX orders_unbilled_index ON orders (order_nr)
    WHERE billed is not true;
```

```
SELECT * FROM orders WHERE billed is not true AND order_nr < 10000;
```

```
SELECT * FROM orders WHERE billed is not true AND amount > 5000.00;
```

Operator Classes

CREATE INDEX name ON table
(column opclass [(opclass_options)] [sort options] [, ...]);

- The operator class identifies the operators to be used by the index for that column.
- For example, a B-tree index on the type int4 would use the int4_ops class; this operator class includes comparison functions for values of type int4.

CREATE INDEX test_index ON test_table (col varchar_pattern_ops);

Operator Classes

Query shows all defined operator classes:

```
SELECT am.amname AS acc_name,
       opc.opcname AS ops_name,
       opr.oprname AS ops_comp
FROM pg_am am, pg_amop amop,
       pg_opclass opc, pg_operator opr
WHERE amop.amopid = am.oid AND
       amop.amopclaid = opc.oid AND
       amop.amopopr = opr.oid
ORDER BY acc_name, ops_name, ops_comp;
```

Examining Index Usage

EXPLAIN SELECT statement;

EXPLAIN ANALYZE SELECT statement;

Choosing the Right Index

Selecting the appropriate index type depends on the nature of your data and the queries you intend to run.

It's essential to consider factors like data distribution, query patterns, and the specific requirements of your application.

Best Practices for Indexing

1. **Identify Query Performance Bottlenecks:**
   Analyze query performance using tools like EXPLAIN and EXPLAIN ANALYZE to identify where indexes can improve query execution.
2. **Avoid Over-Indexing:**
   While indexes can enhance query performance, over-indexing can lead to increased storage overhead and slower write operations. Only create indexes that are necessary for your queries.
3. **Maintain Indexes:**
   Regularly monitor and maintain your indexes to ensure they remain effective. PostgreSQL provides tools like REINDEX and VACUUM to help with maintenance.

Best Practices for Indexing

4. **Consider Multi-column Indexes:**
   For queries that involve multiple columns in the WHERE clause, consider creating multi-column indexes to cover the query's needs.
5. **Use Partial Indexes:** If you have a subset of data that is frequently queried, consider creating a partial index that includes only the relevant rows. This can improve query performance and reduce index size.
6. **Evaluate Index Types:** Carefully choose the index type that best suits your data and query patterns. Experiment with different types and measure their impact on query performance.

Common Index Summary

Index Type	Data types	Operators
B-Tree	Most built in Types	<, <=, =, >, >=, BETWEEN, IN, IS NULL, IS NOT NULL ~ (LIKE: col Like 'Binh Thanh%') ~* (ILIKE: col Ilike 'Binh Thanh%')
Hash	Most built in Types	=
BRIN - Min/Max	Most built in Types	<, <=, =, >, >=
BRIN - Box	box	<< , &<, &&, &>, >> , ~= @>, <@, &<|, <<|, |>>, |&>
BRIN - Range	any range type	<< , &<, &&, &>, >> , @>, <@, -|-, =<, <=, =, >, >=
BRIN - Network	inet	&&, >>=, =, << , >>

Common Index Summary

Index Type	Data types	Operators
GIN - Arraye	any array type	&&, <@, =, @>
GIN - JSONB	jsonb	?, ?&, ?|, @>, @?, @@
GIN - Arraye	any array type	&&, <@, =, @>
GIN - JSONB	jsonb	?, ?&, ?|, @>, @?, @@
GIN - JSONB Path	jsonb	@>, @?, @@
GIST - Text Search	tsvector	@@, @@@
	tsvector	@@
SP-GIST - Text	text	< <, > >, ~=, <@, < <|, |> >
SP-GIST - Range	any range type	&&, &<,&>, -|-, < <, <@, =, > >, @>

Example

B-Tree

B-tree là loại chỉ mục mặc định và linh hoạt nhất, được sử dụng cho hầu hết các kiểu dữ liệu và nhiều loại toán tử.

- **Tìm kiếm chính xác (=):**

  CREATE INDEX idx_user_id ON users USING btree(user_id);

  SELECT * FROM users WHERE user_id = 123;

- **Tìm kiếm phạm vi (<, >, <=, >=):**

  CREATE INDEX idx_order_date ON orders USING btree(order_date);

  SELECT * FROM orders
  WHERE order_date BETWEEN '2025-01-01' AND '2025-01-31';

Example

- **Tìm kiếm theo mẫu (LIKE) trên chuỗi:**

```
CREATE INDEX idx_product_name ON products
  USING btree(name varchar_pattern_ops);

SELECT * FROM products WHERE name LIKE 'Laptop%';
```

Hash

Chỉ mục Hash chỉ hỗ trợ tìm kiếm chính xác (=) và rất nhanh khi cần tìm một giá trị cụ thể.

- **Tìm kiếm chính xác (=):**

```
CREATE INDEX idx_status ON logs USING hash(status);

SELECT * FROM logs WHERE status = 'ERROR';
```

Example

GIN (Generalized Inverted Index)

GIN được thiết kế cho các kiểu dữ liệu phức tạp như mảng, JSONB, và tìm kiếm toàn văn bản.

- **Tìm kiếm trên mảng (@>) :**
  CREATE INDEX idx_tags ON articles USING gin(tags);

  SELECT * FROM articles WHERE tags @> ARRAY['database', 'sql'];
- **Tìm kiếm trong JSONB (?) :**

  CREATE INDEX idx_user_data ON users USING gin(user_data);

  SELECT * FROM users WHERE user_data ? 'age';

Example

- **Tìm kiếm toàn văn bản (@@) :**

```
CREATE INDEX idx_search_body ON documents
  USING gin(to_tsvector('english', body));

SELECT * FROM documents
WHERE to_tsvector('english', body) @@ to_tsquery('english',
    'database & postgres');
```

Example

BRIN (Block Range Index)

BRIN hoạt động hiệu quả trên các bảng lớn với dữ liệu có tính chất sắp xếp tự nhiên (ví dụ: dữ liệu được thêm vào theo thứ tự thời gian).

- **Tìm kiếm phạm vi (<=, >=) :**
  CREATE INDEX idx_log_time ON logs USING brin(timestamp);

  SELECT * FROM logs
  WHERE timestamp > '2025-08-01' AND timestamp < '2025-08-02';

Example

GIST (Generalized Search Tree)

GIST là một cấu trúc cây tìm kiếm tổng quát, cho phép tạo chỉ mục cho nhiều kiểu dữ liệu phức tạp khác nhau.

- **Tìm kiếm toàn văn bản (Text Search):** GIST có thể được sử dụng để tăng tốc các truy vấn tìm kiếm toàn văn bản.

  – Giả sử cột 'content' chứa văn bản cần tìm
  CREATE INDEX idx_content_gist ON documents USING gist (to_tsvector('english', content));

  – Truy vấn tìm kiếm các tài liệu chứa cả hai từ 'database' và 'sql'
  SELECT * FROM documents
  WHERE to_tsvector('english', content) @@ to_tsquery('english', 'database & sql');

Example

- **Tìm kiếm trên dữ liệu hình học (Geometry):** GIST rất mạnh mẽ cho các kiểu dữ liệu hình học như box, circle, polygon trong các hệ thống GIS (Geographic Information System).

  – Giả sử cột 'location' chứa tọa độ
  CREATE INDEX idx_location_gist ON places USING gist (location);

  – Truy vấn tìm kiếm các địa điểm nằm bên trong một hình hộp cụ thể
  SELECT * FROM places
  WHERE location $<<$ '(10,10),(20,20)';

Example

- **Tìm kiếm trên địa chỉ mạng (Inet):**

  – Giả sử cột 'ip_address' chứa địa chỉ IP
  CREATE INDEX idx_ip_address_gist ON access_logs
  USING gist (ip_address inet_ops);

  – Truy vấn tìm kiếm các bản ghi có địa chỉ IP nằm trong một dải mạng
  SELECT * FROM access_logs
  WHERE ip_address << '192.168.1.0/24';

Example

SP-GIST (Space-Partitioned Generalized Search Tree)

SP-GIST là một biến thể của GIST, được tối ưu hóa cho các bài toán phân vùng không gian (space-partitioning problems). Nó thường hiệu quả hơn GIST cho các trường hợp cụ thể.

- **Tìm kiếm trên dữ liệu hình học (Geometry):** Tương tự GIST, SP-GIST cũng rất hiệu quả cho các dữ liệu hình học, đặc biệt khi cần tìm kiếm lân cận gần nhất.

  – Giả sử cột 'geom' chứa dữ liệu hình học
  CREATE INDEX idx_geom_spgist ON buildings
  USING spgist (geom);

  – Truy vấn tìm kiếm các tòa nhà nằm trong một hình hộp cụ thể
  SELECT * FROM buildings
  WHERE geom && '((1,1),(10,10))'::box;

Example

- **Tìm kiếm phạm vi (Range):** SP-GIST cũng có thể được sử dụng cho các kiểu dữ liệu phạm vi (range types).

  – Giả sử cột 'booking_time' là một kiểu dữ liệu daterange
  CREATE INDEX idx_booking_time_spgist ON bookings
  USING spgist (booking_time);

  – Truy vấn tìm kiếm các đặt chỗ bị trùng lặp trong một khoảng thời gian
  SELECT * FROM bookings
  WHERE booking_time && '[2025-08-01, 2025-08-05]';

Overview

Query processing is the set of steps a database system uses to get results from a query.
These steps include:

- **Translating** the SQL query into lower-level operations the system understands
- **Optimizing** the query so it runs more efficiently
- **Executing** the plan to retrieve the actual data from storage

parser $\rightarrow$ *analyzer* $\rightarrow$ *rewriter* $\rightarrow$ *optimizer* $\rightarrow$ *executor*

Overview

• Each SQL query can be turned into one or more relational algebra expressions. These expressions show what the query wants to do, but not exactly how to do it. There are usually many ways to evaluate a relational algebra expression.

• As an illustration, consider the query:

Select Salary
From Employee
Where Salary $<$ 5000 ;

• This query can be translated into either of the following relational-algebra expressions:

- $\sigma_{Salary<5000}(\Pi_{Salary}(Employee))$
- $\Pi_{Salary}(\sigma_{Salary<5000}(Employee))$

Measures of Query Cost

1. Why Measure Query Cost?
   - A single SQL query can have **multiple different execution plans**.
   - The DBMS's **Query Optimizer** is responsible for choosing the plan with the lowest cost.
   - To do this, it needs a **cost model** to compare these plans.
2. Primary Measures of Cost

   In the basic model, the cost of a query is measured by the **total number of disk block accesses** and **CPU cost**.
   - Disk Block Accesses (I/O Cost)
   - CPU Cost
   - Communication Cost

Measures of Query Cost

3. Factors Affecting Cost

- **Relation Size:** The number of tuples and the number of disk blocks a relation occupies. Larger sizes generally lead to higher I/O costs for Seq Scan.
- **Block Size:** The number of tuples that can fit into a block. Larger blocks can reduce the number of disk accesses but increase the amount of unnecessary data read.
- **Indexes:** Significantly reduce I/O and CPU costs for selective search queries by avoiding Seq Scan.
- **Buffer Pool Size:** The amount of RAM dedicated to caching recently accessed disk blocks. A larger buffer pool helps reduce the number of physical disk reads.
- **Data Statistics:** The Query Optimizer uses statistics about data distribution (e.g., number of unique values, histograms) to estimate the number of rows that will be returned from each step. Accurate statistics are crucial for making correct cost estimations.

Measures of Query Cost

4. Cost Estimation for Basic Operations
   - Sequential Scan:
     - Cost = Number of disk blocks of the relation (all blocks must be read).
   - Index Scan:
     - Cost = Cost of reading the index (usually a few blocks) + Cost of reading the actual data blocks (only blocks containing matching records).
     - Much more efficient when the query is highly selective (returns only a small fraction of the data).
   - Join Operations:
     - Nested-Loop Join: Cost depends on the join order and the size of the two relations. Can be very expensive if the inner relation has to be scanned multiple times from disk.
     - Hash Join, Sort-Merge Join: These algorithms have more complex costs, often involving the cost of building hash tables/sorting and I/O costs to read the relations.

Measures of Query Cost

5. Estimating Selectivity and Result Size

   - To estimate costs accurately, the Query Optimizer needs to estimate the selectivity of predicates and the result size of each operation.
   - Selectivity: The proportion of records that satisfy a given condition.
   - Result size estimation: The expected number of records that will be produced by an operation.
   - These estimations rely on data statistics collected by the DBMS (e.g., the ANALYZE command in PostgreSQL).

Overview

Query optimization is the process of **choosing the most efficient execution plan** from many possible ways to process a query - especially when the query is complex.

Instead of expecting users to write optimized queries, the database system **automatically generates an efficient execution plan** that minimizes cost.

Optimization happens at two levels:

- **Relational algebra level:** finding an equivalent but more efficient version of the query.
- **Execution strategy level:** choosing algorithms, using indexes, and determining join orders.

e.g.

instructor(ID, name, deptName, salary)
teaches(ID, courseID, secID, semester, year)
course(courseID, title, deptName, credits)

Overview

• e.g. "Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach."

• Relational algebra expression:

$$\Pi_{name,title}(\sigma_{deptName="Music"}(instructor \bowtie (teaches \bowtie \Pi_{courseID,title}(course)))) \quad \text{(a)}$$

• Subexpression: $instructor \bowtie teaches \bowtie \Pi_{courseID,title}(course)$ in the preceding

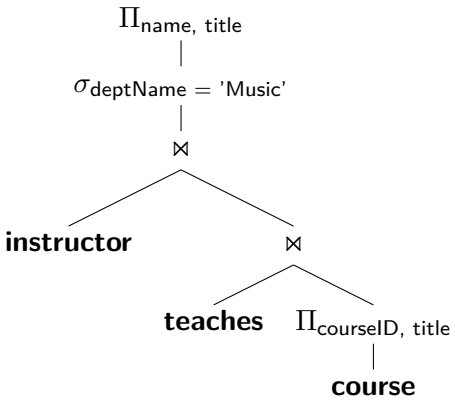
can create a very large intermediate result.

• Because we're only interested in instructors from the Music department, we can ignore all rows where deptName $\neq$ 'Music'.

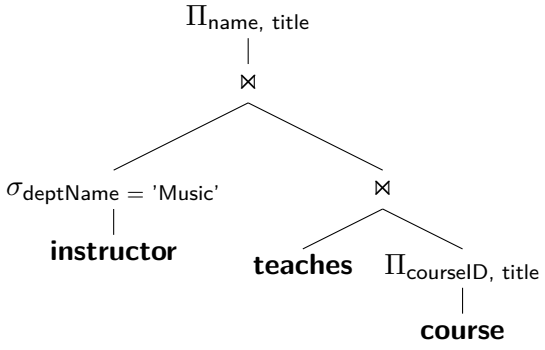
• By doing this early, we reduce the number of rows we need to process later, which means the intermediate result will be smaller and faster to handle.

$$\Pi_{name,title}((\sigma_{deptName="Music"}(instructor)) \bowtie (teaches \bowtie \Pi_{courseID,title}(course))) \quad \text{(b)}$$

Overview



(a) Initial expression tree



(b) Transformed expression tree

Equivalence rules for Relational Algebra

$\theta, \theta_1, \theta_2$: Predicates
L_1, L_2, L_3: Lists of attributes
E, E_1, E_2: Relational algebra expressions
r: Relation.

Equivalence rule: Expressions of two forms are equivalent:

1. **Cascade of σ**

   $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

2. **Commutativity of σ**

   $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\theta_{\theta_1}(E))$

Equivalence rules for Relational Algebra

3. **Cascade of Π**

   $\Pi_{X_1}(\Pi_{X_2}(...(\Pi_{X_n}(E))...)) \equiv \Pi_{X_1}(E)$

   where $X_1 \subseteq X_2 \subseteq ... \subseteq X_n$.

4. σ **combited with** $\times$ **and** $\bowtie_\theta$

   4.1 $\sigma_\theta(E_1 \times E_2) \equiv E_1 \bowtie_\theta E_2$

   4.2 $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

5. **Commutativity of** $\bowtie_\theta$ **and** $\times$

   5.1 $E_1 \bowtie_\theta E_2 \equiv E_2 \bowtie_\theta E_1$

   5.2 $E_1 \times E_2 \equiv E_2 \times E_1$

Equivalence rules for Relational Algebra

6. **Associativity of** $\bowtie$
   - 6.1 $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$
   - 6.2 $(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$

   where θ_2 involves attributes from only E_2 and E_3.

7. σ **distributes over the** $\bowtie_\theta$
   - 7.1 $\sigma_{\theta_1}(E_1 \bowtie_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_\theta E_2$

     where σ_{θ_1} involve only E_1.
   - 7.2 $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_\theta (\sigma_{\theta_2}(E_2))$

     where σ_1 involves only E_1 and σ_2 involves only E_2.

Equivalence rules for Relational Algebra

8. Π **distributes over the** $\bowtie_\theta$
   - 8.1 $\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) \equiv (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$

     where L_1 and L_2 be attributes of E_1 and E_2.
   - 8.2 $\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) \equiv \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$

     where L_1 and L_2 be attributes of E_1 and E_2. L_3 be attributes of E_1 that are involved in $\bowtie_\theta$, but are not in L_1 and L_4 be attributes of E_2 that are involved in $\bowtie_\theta$, but are not in L_2.

   Similar equivalences hold for outer join $⟗$, $⟕$, $⟖$.

9. **Commutativity of Set**
   - 9.1 $E_1 \cup E_2 \equiv E_2 \cup E_1$
   - 9.2 $E_1 \cap E_2 \equiv E_2 \cap E_1$

Equivalence rules for Relational Algebra

10. **Associativity of $\cup$ and $\cap$**
    - 10.1 $(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_2)$
    - 10.2 $(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_2)$
11. **σ distributes over $\cup, \cap, -$**
    - 11.1 $\sigma_\theta(E_1 \cup E_2) \equiv \sigma_\theta(E_1) \cup \sigma_\theta(E_2)$
    - 11.2 $\sigma_\theta(E_1 \cap E_2) \equiv \sigma_\theta(E_1) \cap \sigma_\theta(E_2)$
    - 11.3 $\sigma_\theta(E_1 - E_2) \equiv \sigma_\theta(E_1) - \sigma_\theta(E_2)$
    - 11.4 $\sigma_\theta(E_1 \cap E_2) \equiv \sigma_\theta(E_1) \cap E_2$
    - 11.5 $\sigma_\theta(E_1 - E_2) \equiv \sigma_\theta(E_1) - E_2$

Equivalence rules for Relational Algebra

12. Π **distributes over** $\cup$

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

13. σ **distributes over aggregation of Group**

$$\sigma_\theta({}_G\gamma_A(E)) \equiv {}_G\gamma_A(\sigma_\theta(E))$$

14. **Commutativity of Outer join**

    14.1 $E_1 ⟕ E_2 \equiv E_2 ⟕ E_1$

    14.2 $E_1 ⟗ E_2 \equiv E_2 ⟗ E_1$

15. σ **distributes over** $⟕, ⟗$

    15.1 $\sigma_{\theta_1}(E_1 ⟕_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) ⟕_\theta E_2$

    15.2 $\sigma_{\theta_1}(E_2 ⟗_\theta E_1) \equiv (E_2 ⟗_\theta (\sigma_{\theta_1}(E_1)))$

Equivalence rules for Relational Algebra

16. **Outer joins can be replaced by inner joins**

    16.1 $\sigma_{\theta_1}(E_1 \rtimes_{\theta} E_2) \equiv \sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2)$

    16.2 $\sigma_{\theta_1}(E_2 \ltimes_{\theta} E_1) \equiv \sigma_{\theta_1}(E_2 \bowtie_{\theta} E_1)$

    where θ_1 has false or unknown whenever the attributes of E_2 are null.

Examples of Transformations

e.g University Database:

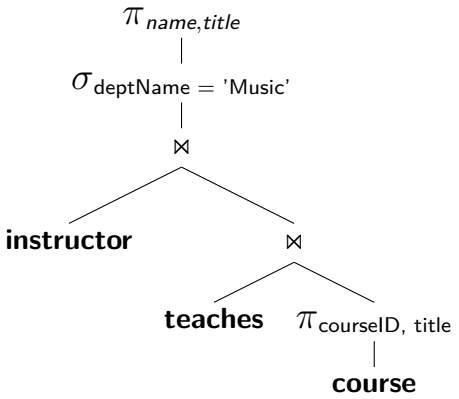
instructor(ID, name, deptName, salary)
teaches(ID, courseID, secID, semester, year)
course(courseID, title, deptName, credits)

expression:

$$\pi_{name,title}(\sigma_{deptName='Music'}(instructor \bowtie (teaches \bowtie \pi_{courseID,title}(course))))$$

Examples of Transformations

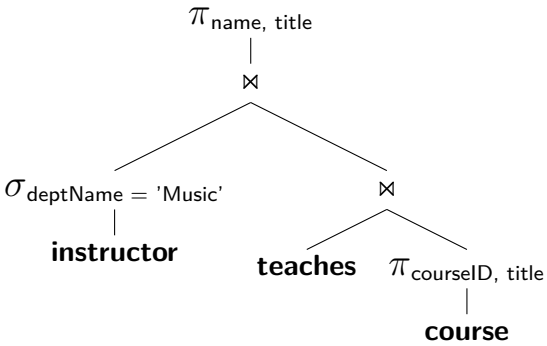
$\pi_{name,title}(\sigma_{deptName='Music'}(instructor \bowtie (teaches \bowtie \pi_{courseID,title}(course))))$



Examples of Transformations

deptName in **instructor**, we transform into the following expression:

$\pi_{name,title}((\sigma_{deptName='Music'}(instructor)) \bowtie (teaches \bowtie \pi_{courseID,title}(course)))$



Examples of Transformations

Suppose that we modify original query: instructors who have taught a course in 2026.

$$\pi_{name,title}(\sigma_{deptName='Music' \wedge year=2026}(instructor \bowtie (teaches \bowtie \pi_{courseID,title}(course))))$$

Using rule 7.1:

$$\pi_{name,title}((\sigma_{deptName='Music' \wedge year=2026}(instructor \bowtie teaches)) \bowtie \pi_{courseID,title}(course))$$

Using rule 1, we can break the selection into two selections:

$$\sigma_{deptName='Music'}(\sigma_{year=2026}(instructor \bowtie teaches))$$

Apply 7.1 and "perform selections early":

$$\sigma_{deptName='Music'}(instructor) \bowtie \sigma_{year=2026}(teaches)$$

Examples of Transformations

